RESTAURANT TABLE BOOKING SYSTEM USING NODE.JS

A Project Report

Submitted in Partial Fulfillment of the

Requirements for the Award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Pawan Surendra Sahu

202302107

Rushikesh Shivaji Patil

202302178

Under the esteemed guidance of

Mr.Aman Singh

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

ZSCT'S THAKUR SHYAMNARAYAN DEGREE COLLEGE

(*Affiliated to University of Mumbai*)

MUMBAI -400101

MAHARASHTRA

2025-2026

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

PRN 2023016401987821 Roll.No. 202302107

PRN 2022016401450042 Roll.No. 202302178

Name of the Students:

I .**PAWAN SURENDRA SAHU**
II. **RUSHIKESH SHIVAJI PATIL**

_

2. Title of the project: RESTAURENT TABLE BOOKING SYSTEM USING NODE.JS

3. Name of the Guide: Mr. Aman Singh

4. Teaching Experience of Guide: 2 year

5. Is This is your first submission: yes ☐ No ☐

Signature of the Student Signature of the Guide

Date: Date:

Signature of the Coordinator

Date:

ZSCT'S THAKUR SHAYAMNARAYAN DEGREE COLLEGE

(Affiliate to University of Mumbai)
MUMBAI-MAHARASHTRA-400101

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, "RESTAURANT TABLE BOOKING SYSTEM USING NODE.JS", is bonafide work of Pawan Surendra Sahu , Rushikesh Shivaji Patil bearing Seat No. (107 & 178) submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHONOLOGY from university of mumbai.

Internal Guide **Coordinator**

External Examiner

Date: **Collage Seal**

ABSTRACT

This project presents TableTime — a smart and efficient Restaurant Table Booking System developed to modernize the dining reservation process for customers, restaurant owners, and administrators. The traditional method of calling or walking in to reserve a table is increasingly unreliable and time-consuming. TableTime addresses this gap by providing a fully automated, web-based platform that allows customers to browse restaurants, view menus, and book tables in advance by selecting their preferred date, time, number of guests, and seating preferences through a clean and intuitive interface.

The platform supports three well-defined user roles. Customers can place bookings and track reservation history. Restaurant Owners can manage their profile, update menus, monitor bookings, and control food item availability through a dedicated dashboard. The Super Administrator holds full platform authority — managing users, assigning roles, blocking accounts, and overseeing the entire system from a centralized admin panel. Core features include secure authentication using JWT and Google OAuth 2.0, real-time table availability management, instant booking confirmation, conflict prevention to avoid double bookings, and automated email notifications via Nodemailer.

The technology stack includes React.js with TypeScript on the frontend for a dynamic user interface, Node.js and Express.js on the backend for scalable RESTful API services, and MongoDB as the database for flexible data management. Cloudinary handles cloud-based image storage for restaurant and food item photographs.

The primary objectives are to reduce waiting times, eliminate double bookings through server-side validation, enhance customer satisfaction, and empower administrators with real-time control. The system is designed with simplicity and accessibility in mind, making it suitable for users with varying technical expertise. TableTime represents a meaningful step forward in digitizing hospitality operations — bridging the gap between customers and restaurants through modern technology and delivering a faster, smarter dining experience for all stakeholders.

Keywords: Restaurant Table Booking, React.js, Node.js, MongoDB, JWT, Google OAuth, Role-Based Access Control, Hospitality Automation, Full-Stack Web Application.

ACKNOWLEDGEMENT

We are truly humbled to present to you our project report on **"Restaurant Table Booking System."** We express our sincere gratitude to **Thakur Shyamnarayan Degree College** who has provided us with a great platform to discover new ideas, explore modern technologies, and helped us establish the confidence to develop independent projects from the ground up.

We are deeply thankful to our project guide **Mr. Aman Singh** for his constant support and guidance throughout the development of this project. He always stood beside us with his tireless efforts, valuable insights, and creative ideas that played a crucial role in making this project successful. His encouragement during every stage of development kept us motivated and focused on delivering our best work.

We are also grateful for the co-operation and valuable suggestions rendered by our H.O.D **Ms. Aishwarya Sedamkar**, whose expert guidance helped us refine our approach and broaden our understanding of the subject. Her support has been instrumental in shaping this project into its final form.

We extend our sincere thanks to all the faculty members and staff of our institution who have directly and indirectly contributed to the successful completion of this project. Their technical inputs, encouragement, and willingness to help whenever needed made a significant difference throughout our journey.

Last but not the least, we would like to express our heartfelt gratitude to our Principal, Parents, and Colleagues for their unwavering support, priceless suggestions, and continuous motivation. Their belief in our abilities gave us the strength to overcome challenges and complete this project with dedication and enthusiasm.

DECLARATION

I hereby declare that the project entitled, **"Restaurant Table Booking System"** done at **Thakur Shyamnarayan Degree College, Kandivali (East), Mumbai-400101**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

I further declare that all the information, data, and results presented in this project report are genuine and have been collected, analyzed, and implemented by our team through our own efforts and research. No part of this project has been copied or reproduced from any other source without proper acknowledgement and reference.

The work presented in this report is original and has been carried out under the supervision and guidance of our project guide **Mr. Aman Singh**. All external references, tools, libraries, and technologies used in the development of this project have been appropriately credited and cited within the report.

I also declare that this project has been developed solely for academic purposes and does not intend to infringe upon any intellectual property rights. The system, its design, and its implementation are intended purely as a learning exercise and a demonstration of technical knowledge acquired during the course of our studies.

We take full responsibility for the authenticity and accuracy of the content presented in this project report and stand accountable for any discrepancies that may be found herein.

Pawan Surenda Sahu

Rushikesh Shivaji Patil

TABLE OF CONTENTS

Sr. No.	Title	PageNo.
1	**INTRODUCTION**	1
1.1	Background	1
1.2	Objective	1
1.3	Purpose, Scope and Applicability	1
1.3.1	Purpose	2
1.3.2	Scope	2
1.3.3	Applicability	3
1.4	Achievements	4
1.5	Organization of Report	4
2	**SURVEY OF TECHNOLOGIES**	5 - 11
3	**REQUIREMENTS AND ANALYSIS**	12
3.1	Problem Definition	12
3.2	Requirements Specification	13
3.3	Planning and Scheduling	14
3.4	Software and Hardware Requirements	15-16
3.5	Preliminary Product Description	17
3.6	Conceptual Models	18-22
4	**SYSTEM DESIGN**	23

4.1	Basic Modules	23
4.2	Data Design	24
4.2.1	Schema Design	24
4.2.2	Data Integrity and Constraints	24
4.3	Procedural Design	25
4.3.1	Logic Diagrams	25
4.3.2	Data Structures	26
4.3.3	Algorithms Design	27
4.4	User Interface Design	28-30
4.5	Security Issues	30-32
4.6	Test Cases Design	33-35
5	**IMPLEMENTATION AND TESTING**	36
5.1	Implementation Approaches	37
5.2	Coding Details and Code Efficiency	38
5.2.1	Code Efficiency	39
5.3	Testing Approach	40
5.3.1	Unit Testing	41
5.3.2	Integrated Testing	42
5.3.3	Beta Testing	43
5.4	Modifications and Improvements	44
5.5	Test Cases	45

6	**RESULTS AND DISCUSSION**	46
6.1	Test Reports	47-48
6.2	User Documentation	49-52
7	**CONCLUSIONS**	53
7.1	Conclusion	53
7.1.1	Significance of the System	53
7.2	Limitations of the System	54
7.3	Future Scope of the Project	55

TABLE OF FIGURES

Sr.No	Title	Page.No
1	Next.js Figure 2.1	5
2	Clerk Figure 2.2	7
3	Strip Figure 2.3	8
4	Gantt Chart 3.1	15
5	Pert Chart 3.2	15
6	ER Diagram 3.3	17
7	Sequence Diagram 3.4	18
8	State Diagram 3.5	19
9	State Diagram 3.6	20
10	Data flow 3.7	21-22
11	Logic Diagram 4.1	25
12	Pictorial View 4.2,4.3	29-30
13	Test Cases Design 4.4,4.5	33-34
14	Coding Details 5.1	39
15	Code Efficiency 5.2	40
16	Unit Testing 5.3,5.4	41
17	Integrated Testing 5.5	42-44

18	Test Cases 5.6	45
19	Unit Testing Results 6.1	46
20	Integration Testing Results 6.2	47
21	Troubleshooting 6.3	52

CHAPTER 1
INTRODUCTION

1.1 Background

In today's digital era, technology plays a vital role in transforming the restaurant industry by enhancing convenience and customer satisfaction. A **Restaurant Table Booking System** developed using **Node.**Js implifies the process of reserving tables online, eliminating the need for manual booking and long waiting times. With the growing popularity of web-based solutions, such systems enable users to check real-time table availability, make instant reservations, and receive booking confirmations seamlessly.

Using technologies such as **Node.js**, **Express.js**, and **MongoDB**, the system ensures fast performance, scalability, and secure data handling. Additionally, the integration of APIs for notifications and payment gateways can further improve the customer experience by offering instant alerts and easy payment options. This automation benefits both restaurant owners and customers by streamlining operations, minimizing errors, and improving service efficiency.

Overall, a Node.js-based restaurant table booking system empowers businesses to deliver a modern, user-friendly, and efficient digital dining experience while reducing manual workload and enhancing overall operational productivity.

1.2 Objective

Our platform provides an advanced **Restaurant Table Booking System** built using **Node.js**, designed to simplify the process of table reservations and restaurant management. It integrates modern web technologies to offer features such as **real-time table availability**, **customer registration**, **booking confirmation**, and **automated notifications**. This system stands out as a comprehensive solution that consolidates all essential restaurant booking operations into a single, user-friendly interface.

By digitizing the reservation process, the platform enhances efficiency for both customers and restaurant administrators. Customers can easily view

available tables, select preferred time slots, and confirm their bookings online, while restaurant staff can manage schedules and availability seamlessly. This integration reduces manual effort, minimizes booking errors, and ensures smooth coordination between customers and management.

Overall, the **Restaurant Table Booking System using Node.js** improves operational productivity, customer satisfaction, and service quality by providing a smart, automated, and efficient way to manage restaurant reservations.

1.3 Purpose, Scope and Applicability

1.3.1 PURPOSE

The **Restaurant Table Booking System** aims to revolutionize the dining experience by offering an online, real-time reservation and management solution built using **Node.js**. This system integrates essential features such as table availability checking, booking confirmation, customer registration, and notification management within a single, cohesive platform. What sets this system apart is its ability to streamline the entire booking process — from reservation to confirmation — in a fast, secure, and user-friendly environment.

By consolidating these capabilities, the system enhances operational efficiency for restaurant administrators while providing customers with a convenient and reliable booking experience. It eliminates the need for manual record-keeping, reduces human error, and optimizes table utilization. Additionally, the system supports scalability and integration with payment gateways or feedback modules, making it adaptable for both small and large restaurants.

Ultimately, the **Restaurant Table Booking System using Node.js** empowers both customers and restaurant staff by simplifying operations, saving time, and improving service quality. It serves as a modern, automated solution that promotes digital transformation in the restaurant industry through efficiency, accessibility, and innovation.

1.3.2 Scope

The **Restaurant Table Booking System using Node.js** aims to provide a complete digital solution for automating restaurant reservations and

management. The system allows customers to conveniently **book tables online**, **check real-time availability**, and **receive instant confirmation** without the need for manual communication. Restaurant administrators can manage bookings, update availability, and view customer details through an interactive dashboard.

The scope of the system includes **user registration and authentication**, enabling secure access for both customers and restaurant staff. It also supports **table management**, **booking modification**, and **cancellation options**, giving users flexibility and control over their reservations. Integrated **email or SMS notifications** can be implemented to keep users informed about booking confirmations or changes.

Furthermore, the system can be extended to include **online payment options**, **feedback modules**, and analytic **reports** to monitor booking trends and customer preferences. Designed using **Node.js** for high performance and scalability, the platform ensures seamless operation even during peak hours.

Overall, the **Restaurant Table Booking System** provides an efficient, user-friendly, and automated platform that enhances restaurant operations, improves customer convenience, and promotes digital transformation in the hospitality sector.

1.3.3 Applicability

The **Restaurant Table Booking System using Node.js** has wide applicability across various sectors within the hospitality industry. It can be implemented in **restaurants, cafés, hotels, and banquet halls** to efficiently manage table reservations, optimize seating arrangements, and enhance customer service. By automating the booking process, the system minimizes manual errors, reduces waiting time, and ensures a smooth dining experience for customers.

For **restaurant managers**, it provides an organized platform to monitor bookings, manage table occupancy, and analyze customer data for business insights. **Customers** benefit from the convenience of checking real-time availability and making reservations anytime through a web or mobile interface. The system's scalability also makes it suitable for **multi-branch restaurants** and **franchise-based businesses**, where centralized management and consistent service are crucial.In addition, the platform can

be adapted for use in **event management**, **conference halls**, and **canteens**, where seat or space reservations are required. With integration of **online payment**, **notification services**, and **feedback modules**, the system enhances communication, accountability, and overall service quality.

Overall, the **Restaurant Table Booking System** serves as a versatile solution that promotes digital transformation, improves efficiency, and delivers convenience to both customers and service providers across multiple domains in the hospitality sector.

1.4 Achievement

I worked with several modern technologies that helped me build a complete web application. Using **React** and **Next.js**, I created interactive and fast user interfaces. React allowed me to build reusable components, while Next.js improved the performance of the app by rendering pages on the server and generating static
pages.

For the backend, **Node.js** helped me manage server tasks and handle requests. **Prisma** made it easier to interact with thedatabase, and I used **MongoDB** to store and organize data effectively. **JavaScript** helped me write more reliable code by catching errors early and making the code easier to understand.

I also integrated a payment system to process transactions securely. **Axios** was useful for making and managing HTTP requests to and from other services. Finally, **Shadcn-UI** provided a set of pre-designed components that helped me create a consistent and attractive user interface.
This project gave me hands-on experience with both front-end and back-end development, enhancing my skills and preparing me for future web development challenges.

1.5 Organization of Report

We'll cover two main topics. First, Requirements Analysis involves identifying what the project needs to do and how it should perform. This includes gathering detailed needs from users and documenting them clearly.

Second, the Survey of Technologies is about reviewing and choosing the right tools and technologies for the project. We'll look at different options, consider their pros and cons, and select the best fit based on project needs. This helps ensure the final product works well and meets user expectations.

CHAPTER 2
SURVEY OF TECHNOLOGIES

Next.js is a powerful framework for building React applications. It offers features like
server-side rendering (SSR) and static site generation (SSG). Server-side rendering means that pages are pre-built on the server before they reach the user's browser. This leads to faster load times and better performance. Static site generation pre-builds pages at build time, so they load very quickly when users visit your site. Next.js also makes it easy to handle routing (how users navigate through your site) and to set up API routes, which means you can build server-side functionality right into your frontend project. This all contributes to a smoother, faster, and more efficient web experience.

React is a JavaScript library used to build the user interface of your application. It allows you to create reusable components, which are pieces of code that can be used multiple times across your app. This modular approach makes your code easier to manage and maintain. React's virtual DOM (a lightweight copy of the actual DOM) helps update the user interface quickly and efficiently by only changing the parts of the page that need to be updated, rather than reloading the entire page. This results in a more responsive and dynamic user experience.

Figure 2.1

Magic UI is a user interface tool used in our project to make designing and managing our application easier. It provides an intuitive, user-friendly way to create and customize layouts without needing extensive coding knowledge. With Magic UI, you can quickly drag and drop elements, adjust settings, and preview changes in real time. This helps us speed up development and ensure our app looks great and works well on different devices. Its simplicity and flexibility allow our team to focus on creating a better experience for users while streamlining the design process.

Shadcn UI is a component library that offers pre-designed and customizable user interface elements. These components include buttons, forms, and other UI elements that are ready to use. By using Shadcn UI, you save time on design and development, as you don't need to create these elements from scratch. The library ensures that your app has a consistent and modern look, which helps in creating a polished and professional user experience.

Tailwind CSS is used to design the user interface efficiently. Tailwind CSS is a utility-first framework that provides low-level utility classes for building custom designs quickly. It allows us to apply styles directly in our HTML or JSX, making it easy to create responsive, modern layouts without writing custom CSS from scratch. This approach ensures consistency across the application and speeds up development by reducing the need for custom stylesheets. Tailwind CSS's flexibility and ease of use complement the other technologies in our stack, contributing to a polished and cohesive user interface.

Node.js is a runtime environment that allows you to run JavaScript code on the server side. This is important because it means you can use JavaScript for both the frontend and backend of your application, simplifying the development process. Node.js is designed to handle many requests simultaneously without slowing down, thanks to its non-blocking I/O (input/output) model. This makes it ideal for building scalable applications that need to handle a lot of traffic or data.

TypeScript TypeScript is a superset of JavaScript that adds static typing to the language. It helps detect errors during development by defining data types for variables and functions. This improves code reliability, readability, and maintainability. In this project, TypeScript is used in the frontend to create a structured and error-free Restaurant Table Booking system.

Mongoose is a library that works with MongoDB, a NoSQL database. Mongoose provides a way to define schemas, which are like blueprints for your data. This helps in organizing and managing the data your application uses. With Mongoose, you can easily perform common database operations like creating, reading, updating, and deleting data. It makes working with MongoDB more straightforward by providing a structured approach to interacting with your database.

MongoDB is a flexible database that stores data in a format similar to JSON (JavaScript Object Notation). This format is easy to work with and allows for flexible data structures. Unlike traditional relational databases, which use tables and rows, MongoDB uses collections and documents, making it easier to handle unstructured or semi-structured data. This flexibility is useful for applications that need to adapt to changing data requirements or deal with a variety of data types.

JWT (JSON Web Token) Authentication is a secure method used to verify and authorize users in a web application. It works by generating a token after successful login, which is then used to access protected routes. JWT ensures secure communication between the client and server. In this project, JWT is implemented to protect user accounts and booking data in the Restaurant Table Booking system.

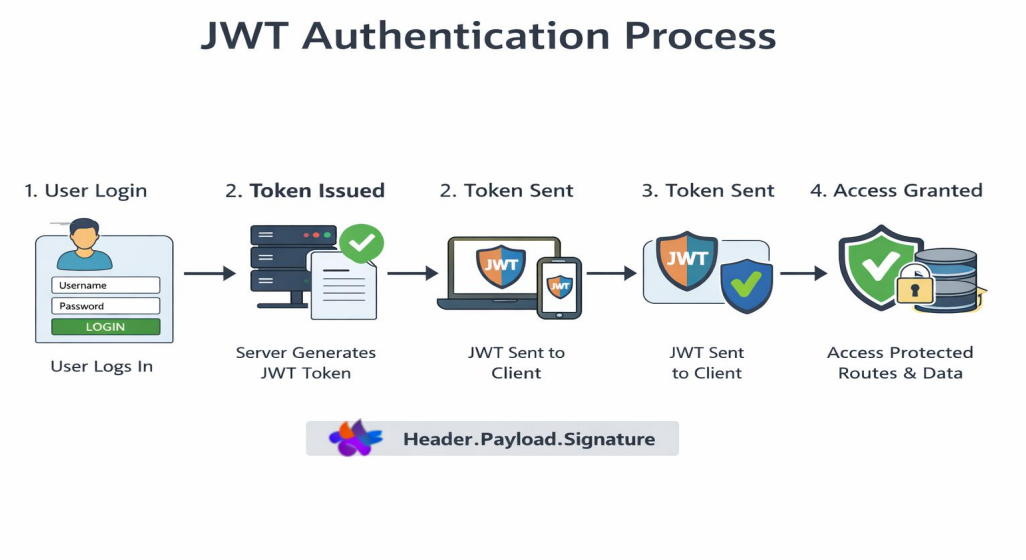


Figure 2.27

Axios is a library used to make HTTP requests from your application. HTTP requests are how your application communicates with servers and other web services. Axios simplifies this process by providing a clean, easy-to-use interface for sending requests and handling responses. It automatically parses JSON data, handles errors, and allows you to cancel requests if needed. This makes it easier to integrate your app with APIs and manage data exchanges.

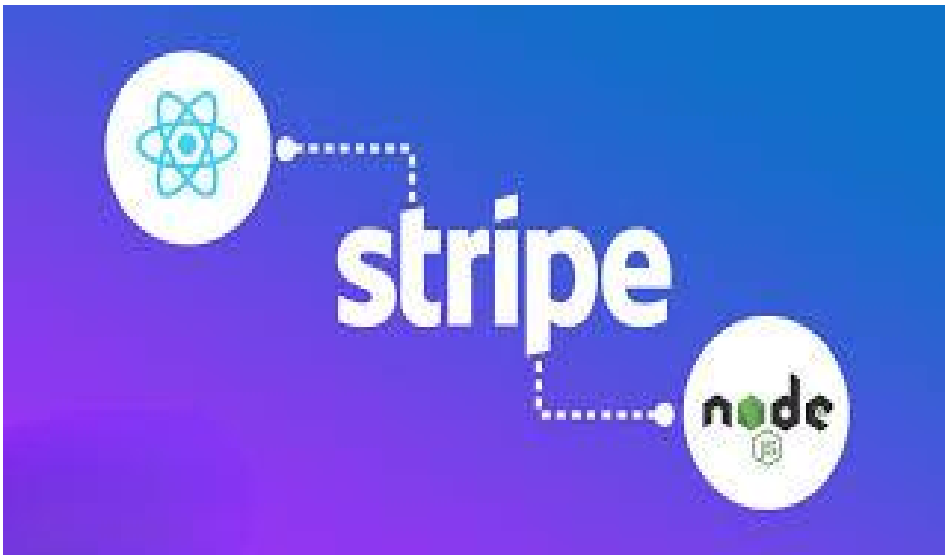


Figure 2.3

Stripe for handling payments. Stripe is a popular payment processing service that makes it easy to accept and manage transactions online. It supports various payment methods, including credit cards and digital wallets, and provides a secure, user-friendly interface for processing payments. Integrating Stripe allows us to handle payments efficiently, manage subscriptions, and issue refunds with ease. Its robust APIs and straightforward setup ensure that our payment system is reliable and easy to maintain, enhancing the overall user experience by providing a seamless and secure payment process.

we explored the technological choices underpinning our project, highlighting how each technology contributes to a streamlined and efficient development process. For the frontend, **Next.js** and **React** were selected for their ability to enhance performance and maintainability. Next.js provides server-side rendering and static site generation, leading to faster load times and

improved SEO. React's component-based architecture allows for reusable and manageable code, which is crucial for building dynamic and interactive user interfaces.

On the backend, **Node.js** was chosen for its efficient handling of server-side JavaScript, promoting a unified development experience across the stack. Coupled with **TypeScript**, which introduces static typing to catch errors early, this combination enhances code reliability and maintainability. **Mongoose** facilitates structured interactions with **MongoDB**, a flexible NoSQL database that excels in handling diverse and scalable data requirements.

For user authentication, **JWT** provides a comprehensive solution with pre-built components, simplifying secure login and registration processes. **Axios** is utilized for API requests due to its ease of use and robust features like automatic JSON parsing and error handling. Additionally, **Magic AI** introduces advanced, intelligent features for personalized user experiences, while **Shadcn UI** offers a library of pre-designed components, speeding up development and ensuring a consistent, modern user interface.

In conclusion, the chosen technologies collectively provide a powerful and cohesive framework for developing a high-performance, scalable application. They not only streamline the development process but also enhance the user experience through efficient data management, secure authentication, and advanced UI features.This integrated approach ensures that both the frontend and backend are optimized for performance, scalability, and maintainability, setting a strong foundation for the success of the project.

Technology Stack Comparison – Preferred Choices, Reasons, and Advantages

Technology	Preferred Choice	Reason	Compared to	Advantages
Next.js	Yes	Provides server-side rendering and static site generation.	Standard React	Improves performance and SEO; simplifies routing and API setup.
React	Yes	Component-based architecture for reusable, maintainable UI.	Angular, Vue.js	Large community, extensive ecosystem, and efficient updates with virtual DOM.
Node.js	Yes	JavaScript runtime for server-side operations, promoting a unified language stack.	Java, PHP	Faster performance for I/O-heavy tasks; easier integration with modern frontend technologies.
TypeScript	Yes	Adds static typing to JavaScript, catching errors early.	JavaScript	Improves code reliability, maintainability, and tooling support.
Mongoose	Yes	Provides schema-based data modeling for MongoDB.	Native MongoDB driver	Simplifies data validation and management with a clear API.

MongoDB	Yes	Flexible NoSQL database for diverse and scalable data storage.	SQL Database	Allows for easy schema changes and scalable data management
Clerk	Yes	Pre-built authentication components for secure user management	Custom-built solutions	Speeds up development and ensures secure, reliable authentication.
Axios	Yes	Simplifies HTTP requests and response handling with built-in features.	Native fetch API	Easier to use with automatic JSON parsing and error handling
Shadcn UI	Yes	Pre-designed UI components for faster and consistent design.	Custom-designed components	Speeds up development and ensures a cohesive, modern look
Stripe	Yes	Reliable payment processing with support for various payment methods.	Other payment processors	Easy to integrate, secure, and user-friendly with strong support for managing transactions.

Table 2.1

This table provides a clear comparison of why each chosen technology is preferred over other

CHAPTER 3
REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Managing restaurant reservations manually can often be time-consuming and error-prone. Customers typically have to call or visit the restaurant to check table availability, which may lead to double bookings, long waiting times, and poor customer satisfaction. Restaurant staff also struggle to maintain accurate booking records, especially during peak hours, leading to mismanagement and loss of potential customers.

Overall Problem There is currently no efficient, centralized platform that allows customers to book restaurant tables online while enabling restaurant managers to manage reservations easily. This lack of automation results in communication gaps, delays, and difficulties in maintaining accurate records of bookings and availability.

Sub-Problems

1. Inefficient Reservation Management Many restaurants still rely on manual methods such as phone calls or physical registers to handle reservations. This manual process increases the chances of human error, duplicate bookings, and confusion during busy hours. It also requires constant staff attention, reducing their ability to focus on customer service.

2.Lack of Real-Time Availability
Customers often cannot view real-time table availability and must rely on staff confirmation. This lack of instant feedback leads to frustration, as users may attempt to book tables that are already reserved. Without an automated system, restaurants struggle to manage occupancy efficiently.

3. Communication Gaps
Since bookings are not automated, customers may not receive immediate confirmation or updates regarding their reservations. Changes or cancellations are difficult to manage and communicate promptly, resulting in inconvenience for both customers and restaurant staff.

4. Limited Data Management
   Without a digital system, restaurant owners find it difficult to maintain historical data about customers, booking frequency, or peak time patterns. This limits their ability to analyze performance, forecast demand, and make informed business decisions.

5. Poor Customer Experience
   Manual booking systems lack convenience and accessibility. Customers expect quick, online solutions for everyday activities. The absence of an easy-to-use online booking platform can negatively impact customer satisfaction and reduce repeat visits.

3.2 Requirement Specification

Our platform is designed to solve common problems that people face when booking tables at restaurants. Right now, many customers have to call or visit restaurants to check table availability and make reservations. This can be time-consuming, prone to errors, and sometimes leads to double bookings. Our goal is to bring all these features into one digital platform, making the booking process faster, easier, and more reliable.

First, the system will allow customers to **check real-time table availability** and **make reservations online** for a specific date and time. It will also let them **modify or cancel bookings** without needing to contact the restaurant manually. This makes it a one-stop solution for all reservation needs.

We're also focusing on **accessibility and convenience**. Customers will receive instant **confirmation via email or SMS**, ensuring they are always informed about their reservations. Unlike traditional methods that provide no real-time updates, our platform will keep both customers and restaurant staff fully synchronized, improving the overall experience.

Another important feature is **administrative management**. Restaurant staff can view all bookings, manage table assignments, and monitor customer history through a centralized dashboard. This saves time, reduces errors, and helps restaurants operate more efficiently.

Collaboration and communication are key, too. The system ensures smooth interaction between customers and staff, making it easy to handle special requests, cancellations, and updates instantly. By solving problems like slow

manual booking processes, poor communication, and lack of real-time updates, our platform makes restaurant reservations quicker, more accessible, and more convenient for everyone.

3.3 Planning and Scheduling

Planning and scheduling are crucial steps in ensuring the successful development of our platform. Proper planning allows us to break down our main goal into smaller, manageable tasks, such as designing the user interface, building the content creation tools, and integrating speech and translation features. By identifying these tasks early on, we can focus on each specific part of the project and know what needs to be done. It's also important to consider any constraints, such as deadlines, available resources, and potential obstacles, so we can manage how these factors might impact the timing of tasks.

Once the planning is in place, scheduling helps us organize everything efficiently. This involves determining what resources are needed for each task, like assigning team members or using specific software. Proper scheduling ensures that the tasks are arranged in a way that avoids bottlenecks and makes the best use of time and resources. By setting clear timelines for when each task should be completed, we can stay on track and make steady progress toward the final goal.

There are many benefits to proper planning and scheduling. It helps keep the project moving forward and ensures that we meet deadlines. It also clarifies each team member's responsibilities, so everyone knows what they need to work on. This improves communication and coordination among the team, reducing confusion or overlap in tasks. Overall, well-organized planning and scheduling are essential for managing the platform's development, as they create a clear roadmap to follow and help avoid delays, ensuring the project is completed on time and within scope

Gantt Chart

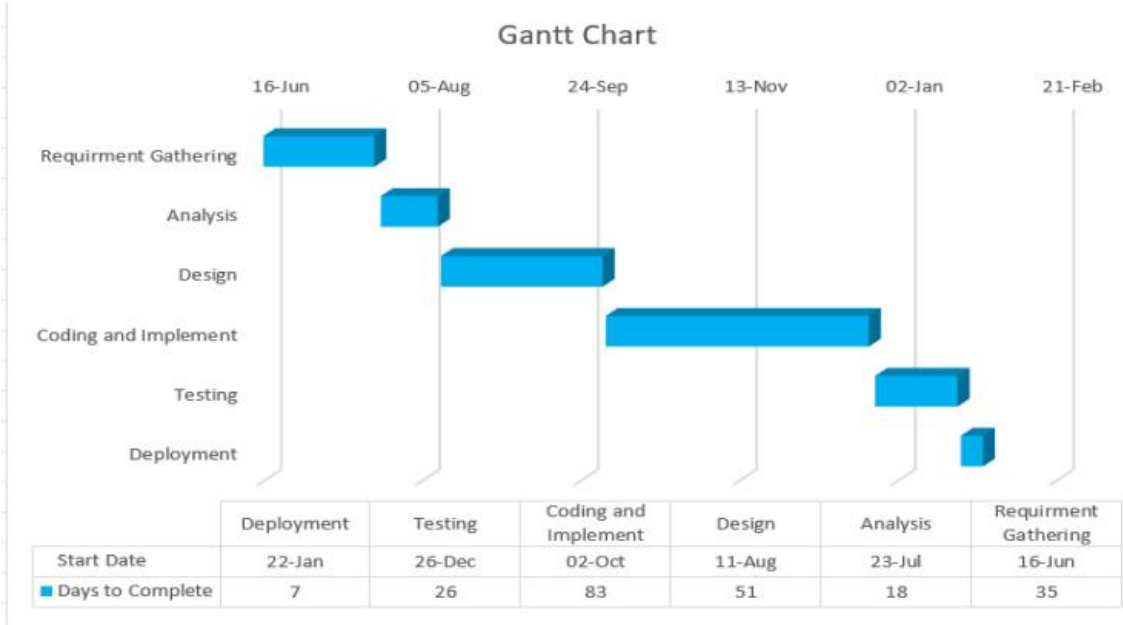


Figure 3.1

PERT Chart

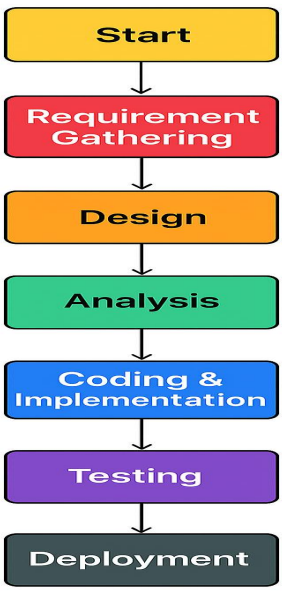


Figure 3.2

3.4 Software and Hardware Requirements

Hardware Requirements To develop and run our platform, certain hardware is essential. A modern PC or laptop with a good processor, such as an Intel i5 or equivalent, is necessary to ensure smooth coding and testing. The system should have at least 8 GB of RAM, allowing it to handle multiple applications running at the same time without lagging. For storage, a 256 GB SSD is recommended for faster file access and enough space for software and project files. Additionally, a standard mouse and keyboard are required for comfortable coding and input during development.

Software Requirements On the software side, several tools and libraries are required to build the platform. An operating system like Windows, macOS, or Linux will be needed to run the development tools. The platform will be built using Next.js, a framework based on React, which will be used to design the user interface. Shadcn UI will provide pre-designed UI components, making it easier to create visual elements quickly. Node.js will serve as the runtime environment for running JavaScript on the server, while TypeScript will help ensure more reliable code by adding static types to JavaScript. MongoDB, a flexible database system that stores data in a JSON-like format, will be used for managing the platform's data. Visual Studio Code will serve as the code editor for writing and editing the platform's code. Lastly, various testing tools will be used for unit and integration testing to ensure the application functions correctly.

3.5 Preliminry Product Description

The proposed system aims to develop a **Restaurant Table Booking Application** that simplifies and digitizes the process of reserving tables at restaurants. The main objective of the system is to provide a convenient, efficient, and user-friendly platform where customers can book tables in advance, avoiding long waiting times and ensuring a smooth dining experience.

The application will allow users to **browse available restaurants**, view detailed information such as location, menu, ratings, and available time slots, and then **book a table** based on their preferred date and time. Registered users will be able to **log in, view their booking history, modify or cancel reservations**, and receive instant booking confirmations via email or SMS.

Restaurant owners or managers will have access to an **admin dashboard** to manage bookings, update table availability, and monitor customer reservations in real time. The system will also include **payment integration**, enabling customers to make advance payments or deposits securely.

By integrating booking, management, and notification features into a single digital platform, the **Restaurant Table Booking System** will enhance customer satisfaction, reduce manual workload for restaurant staff, and promote a more organized and efficient reservation process, ultimately improving overall restaurant management and customer experience.

3.6 Conceptual Model

ER Diagram

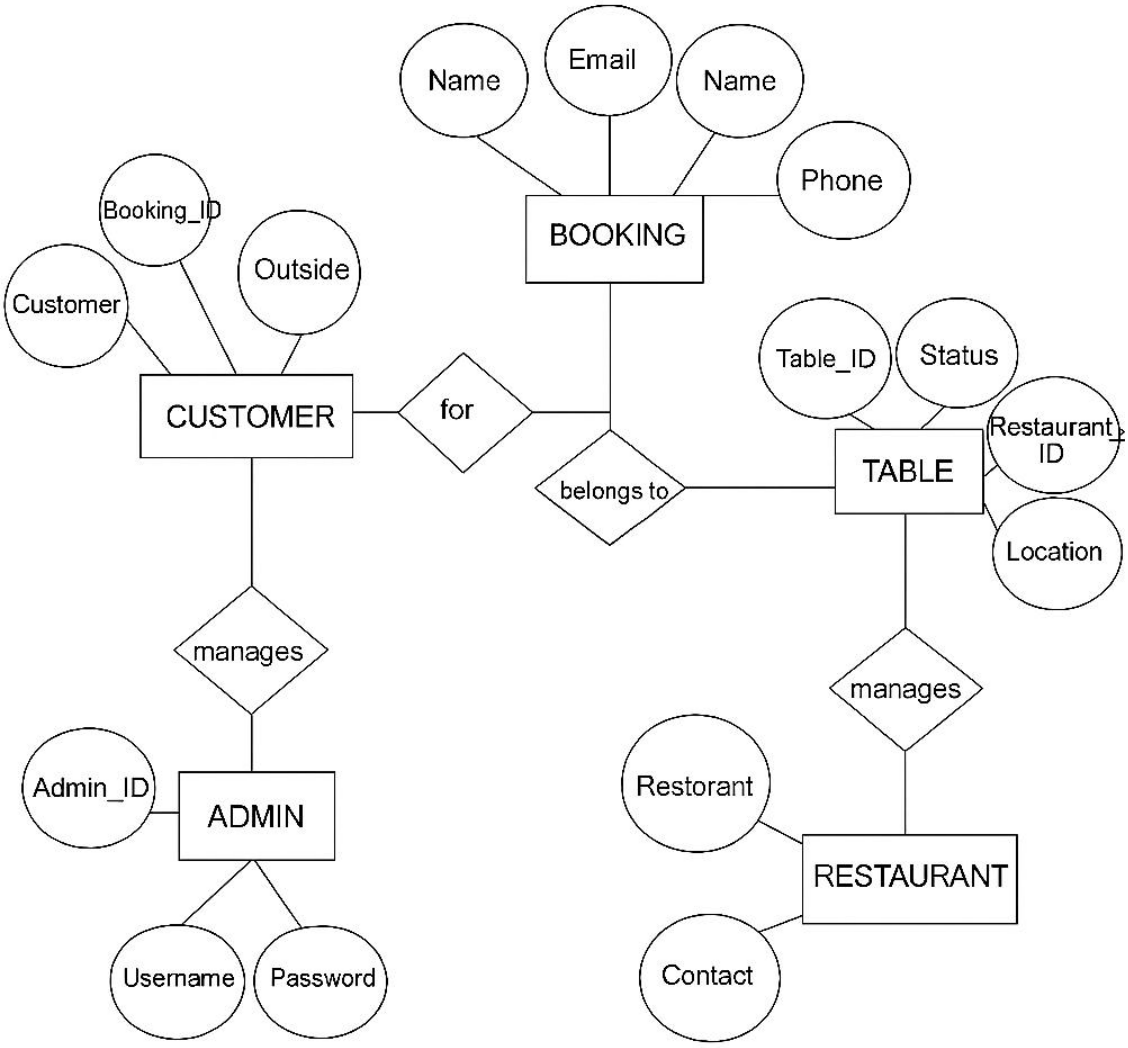
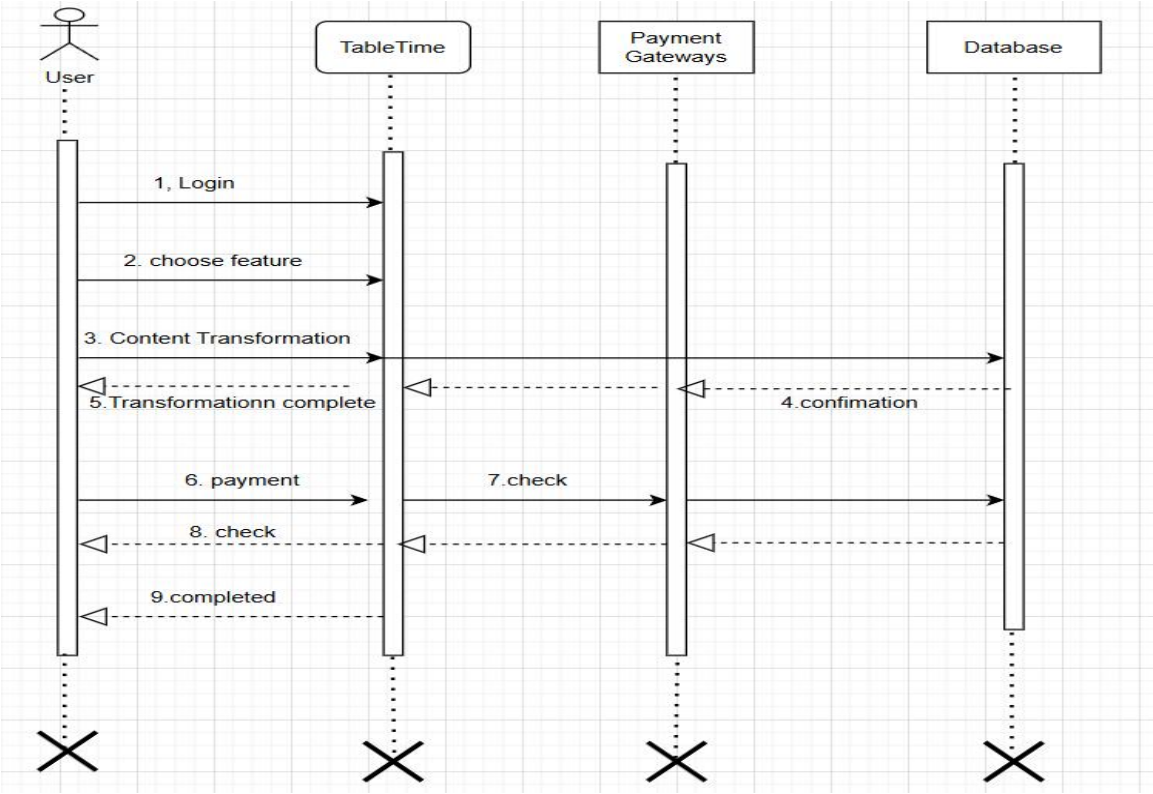


Figure 3.3

Sequence Diagram



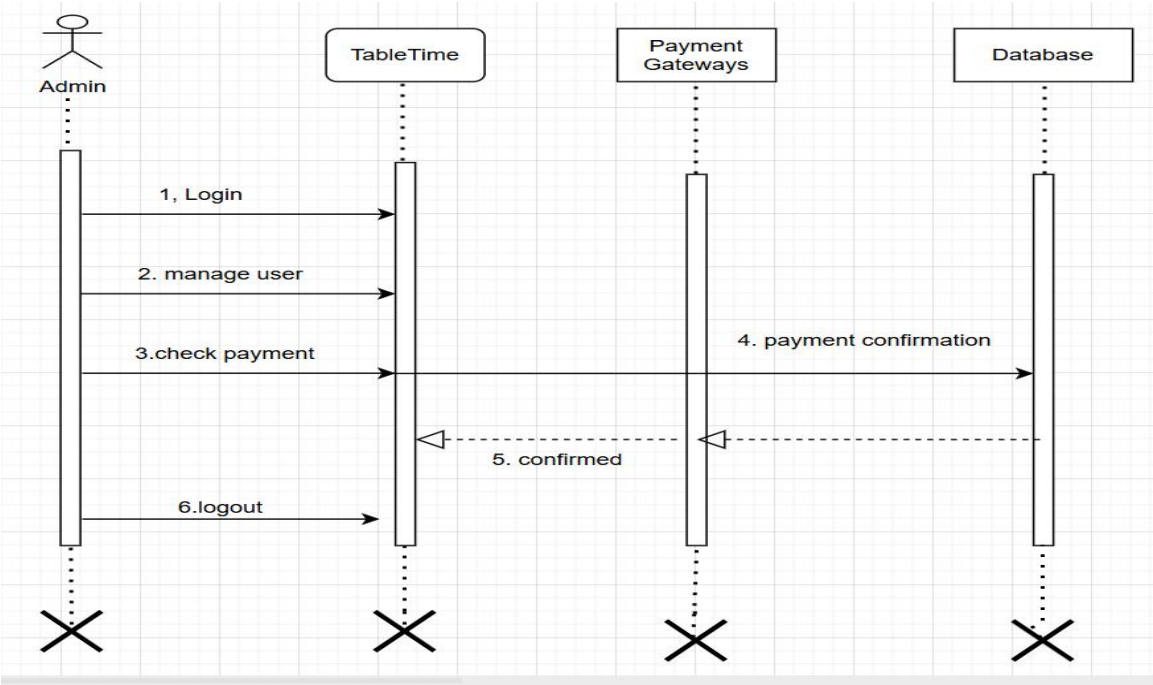


Figure 3.4

State Diagram

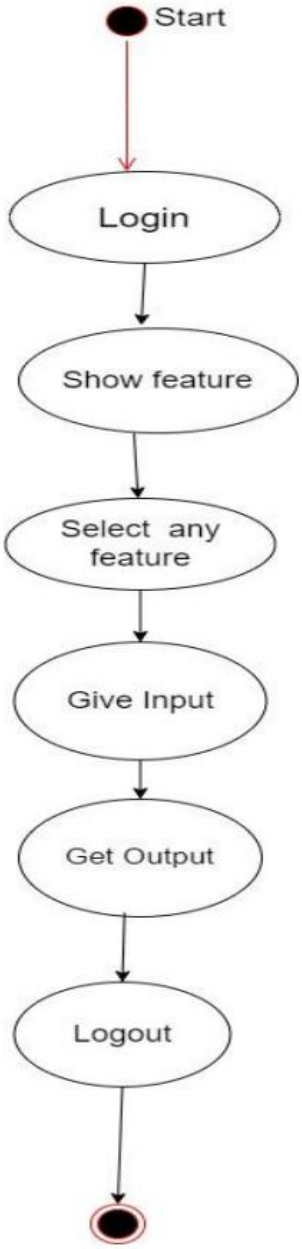


Figure 3.5

Use case Diagram

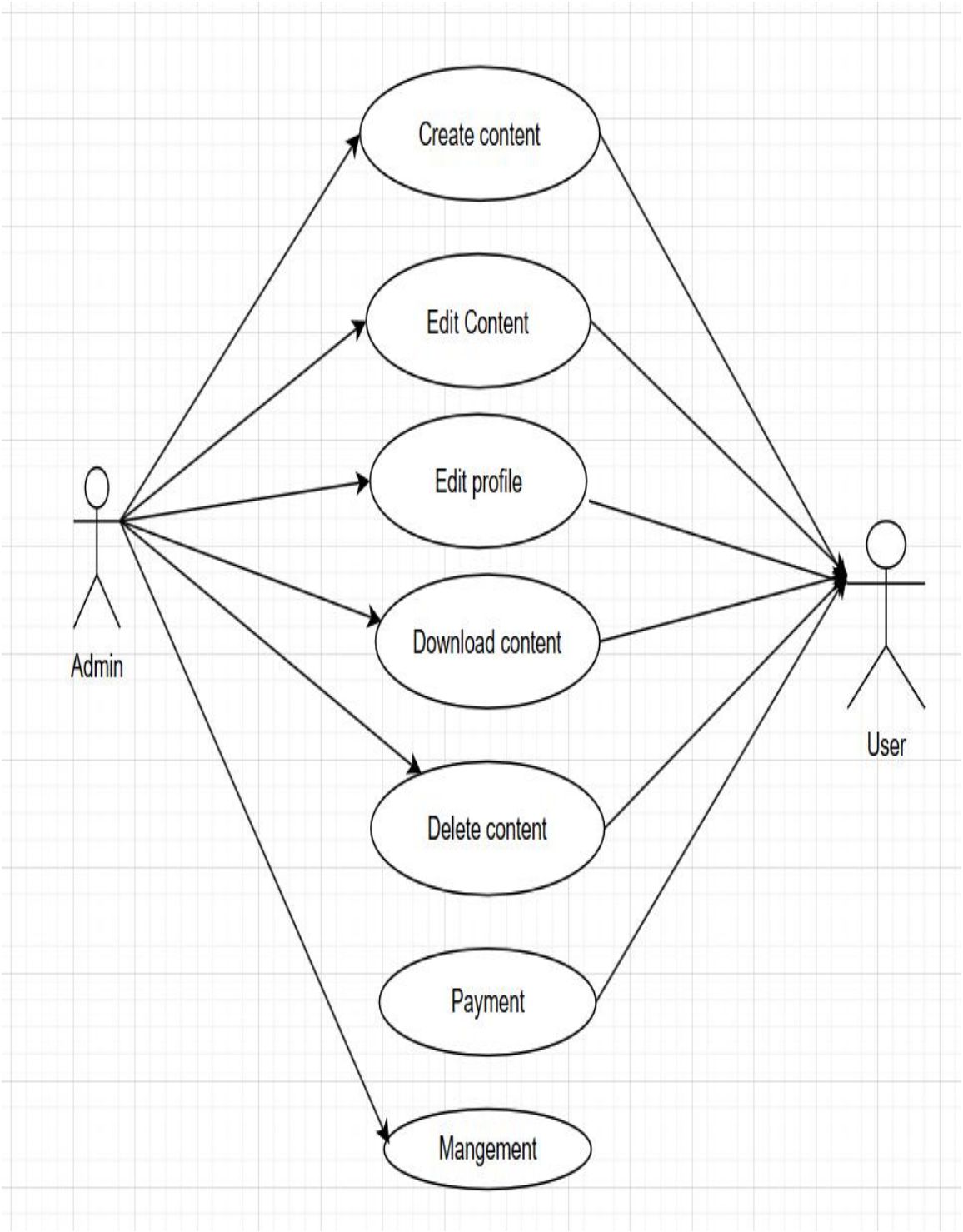
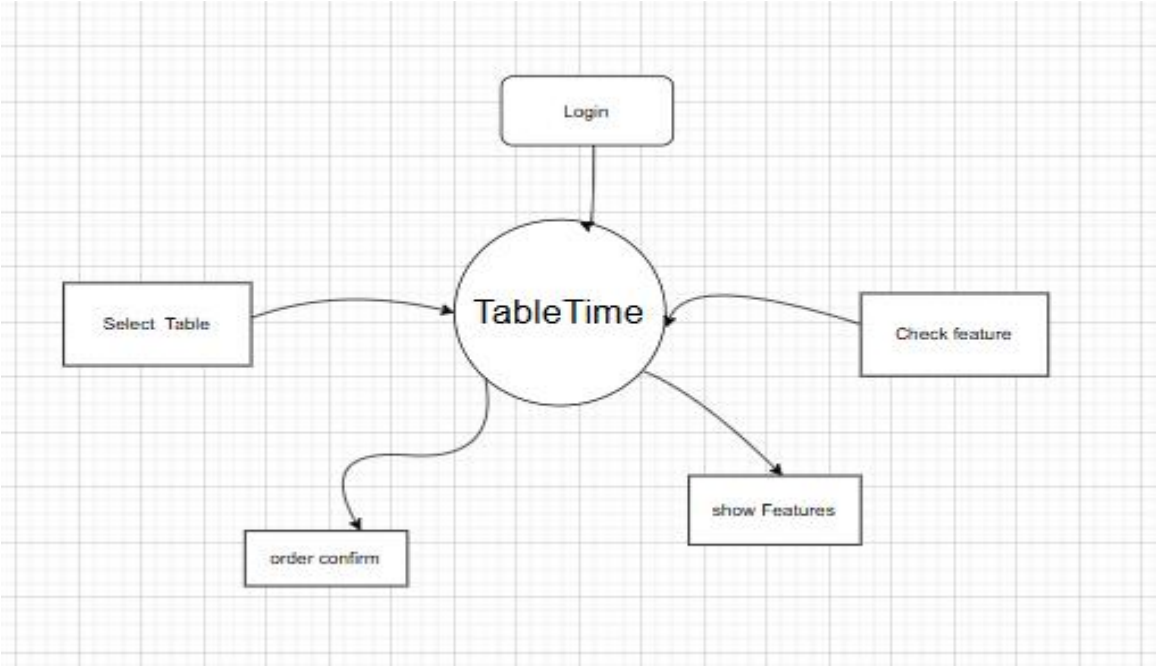


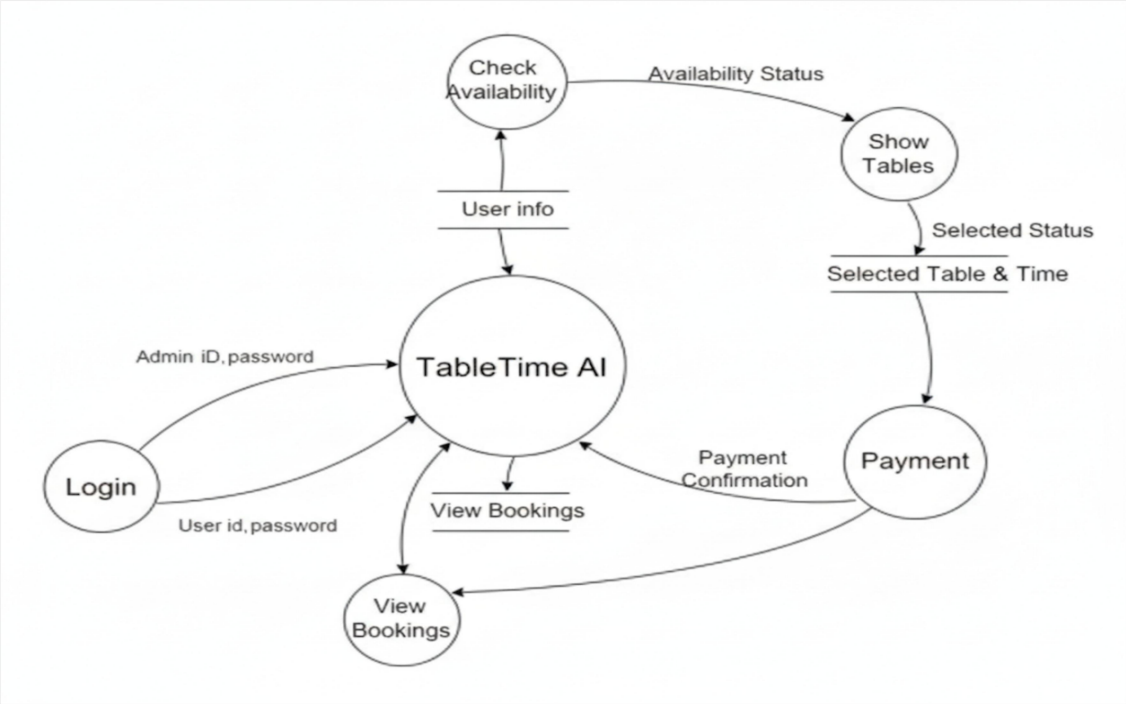
Figure 3.6

Data flow diagram

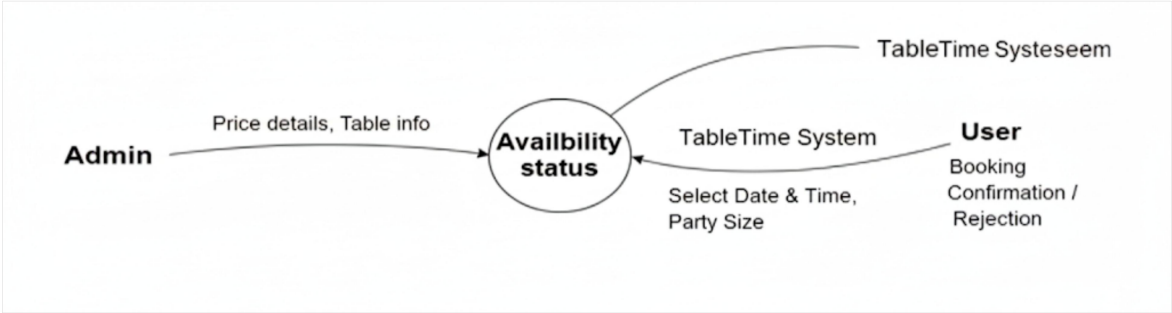
Level 0

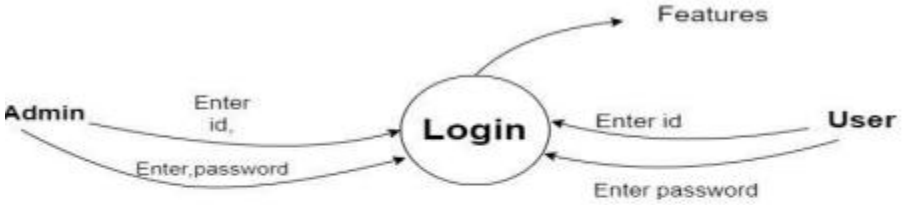


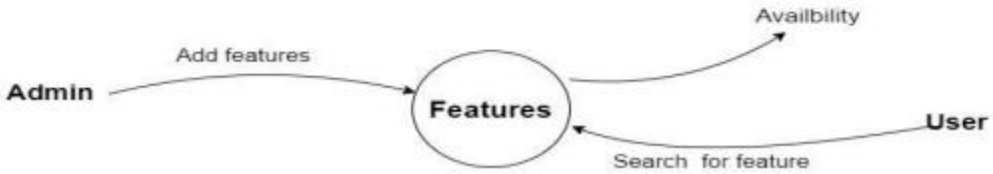
Level 1

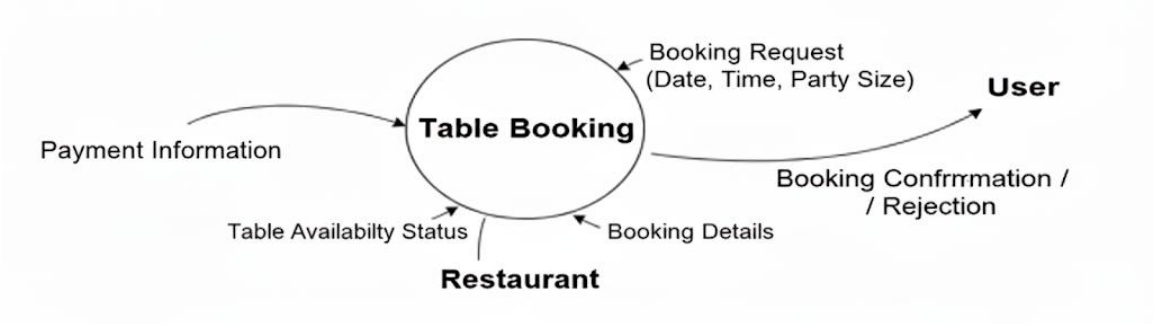


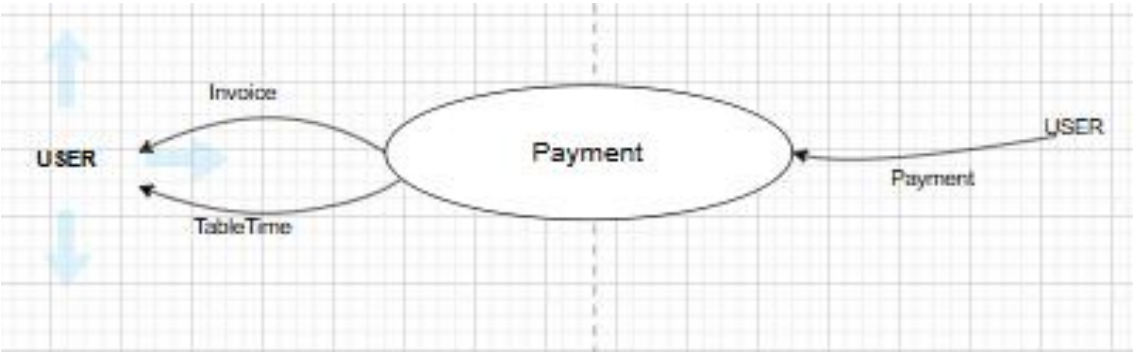
Level 2


TableTime Systeseem
Price details, Table info
Admin
Availbility status
TableTime System
User
Booking Confirmation / Rejection
Select Date & Time, Party Size


Features
Admin
Enter id,
Login
Enter id
User
Enter,password
Enter password


Availbility
Add features
Admin
Features
User
Search for feature


Booking Request
(Date, Time, Party Size)
User
Payment Information
Table Booking
Booking Confrrmation /
/ Rejection
Table Availabilty Status
Booking Details
Restaurant


Invoice
USER
Payment
USER
Payment
TableTime

CHAPTER 4

SYSTEM DESIGN

4.1 Basic Module

To build our **Restaurant Table Booking System using Node.js**, we will break the project into several key parts, each with its own specific role.

The **User Management Module** allows customers to register, log in, and manage their accounts securely. It ensures only authorized users can access the booking features, providing a personalized and safe experience.

The **Table Booking Module** enables users to view available tables and book them by selecting the date, time, and number of guests. It helps manage real-time availability and avoids double bookings, making the reservation process quick and reliable.

The **Admin Management Module** is designed for restaurant staff to manage all bookings, add or remove tables, and monitor customer activity. This keeps the restaurant operations organized and efficient.

The **Notification Module** sends booking confirmations and reminders to users via email or SMS, keeping them informed and reducing missed reservations. The **Analytics Module** tracks booking trends, peak hours, and customer preferences, helping the restaurant make data-driven decisions to improve service and efficiency.

The **Language Support Module** allows users to interact with the system in their preferred language, breaking down communication barriers and improving accessibility for a wider audience.

Finally, the **Security and Session Handling Module** protects user data and sessions using encryption and secure login systems, ensuring a trustworthy and safe environment for all users. This structured modular approach ensures that the Restaurant Table Booking System is user-friendly, secure, and efficient, helping both customers and restaurant staff manage table reservations with ease

4.2 Data Design

Data design is really important for our platform because it decides how we organize and use our data. When we have a good data design, everything works better. Users can easily find the information they need, which makes using the platform easier. By planning how data flows and is stored, we create a solid base that supports all the different features of our platform.

4.2.1 Schema Design

To help us organize our data, we will create different schemas. Think of a schema as a blueprint that shows how our data is structured and connected. For example, we will have one schema for user information, which will include details like name, email, password, and user roles (such as customer or admin). This helps us manage user accounts and login functionality. We will also create another schema for **table data**, which will store information about each table in the restaurant — such as table number, seating capacity, and availability status. In addition, a **booking schema** will be designed to handle reservation details.

This will include the user ID, table ID, booking date and time, number of guests, and booking status. These schemas work together to keep everything well-organized and connected. By clearly defining our schemas, we ensure that the system can store, retrieve, and manage data efficiently, making the entire table booking process smooth and reliable.

4.2.2 Data Integrity and Constraints

To keep our data accurate and reliable, we will set up various checks and rules. These rules are essential for making sure the data in our system is of good quality. For instance, we will ensure that each user has a unique email address. This prevents two users from having the same email, which could cause confusion. Also, some fields, like a user's name or the title of their content, must be filled out before they can submit any forms. This helps stop incomplete information from being saved. Additionally, we will check that any files users upload, such as images or documents, meet specific requirements, like the type of file and its size. By putting these checks and rules in place, we can reduce mistakes and make sure our data stays

trustworthy. Good data design will also improve the experience for users. It will be easier for them to create and manage content on our platform. Overall, careful planning of our data design ensures that our system works well and meets the various needs of all users. This is essential for making the platform user-friendly and effective.

4.3 Procedural Design

Procedural design is a structured method for creating clear instructions or algorithms that outline how tasks are performed in a system. This approach ensures that processes are logical and easy to understand, making it simpler for everyone involved in the project to follow necessary steps.

4.3.1 Logic diagrams

Our visual tools that illustrate the step-by-step flow of procedures, helping to clarify the sequence of actions. These diagrams are especially useful during implementation for troubleshooting and making adjustments. Overall, procedural design and logic diagrams lead to a well-organized system, improving workflows and communication among team members, ensuring our platform runs smoothly.

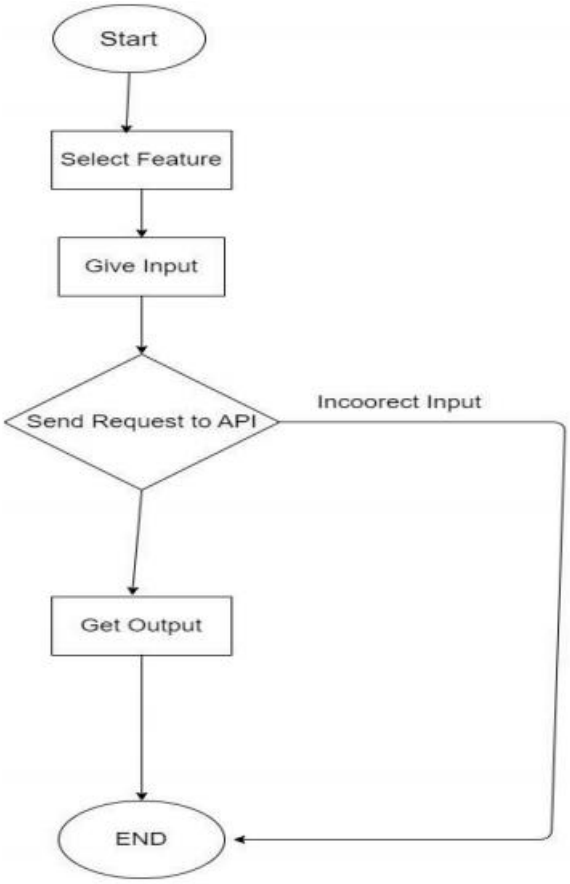


Figure 4.1

4.3.2 Data Structures

In the **Restaurant Table Booking System using Node.js**, data structures are essential for organizing and managing various types of information effectively. Here's a breakdown of the key concepts:

User Data: This structure stores all the information related to users, including customers and administrators. Typical attributes include user ID, name, email, password (stored securely), contact number, and user role (e.g., admin or customer). This data is critical for authentication, authorization, and personalizing the user experience.

Table Data: This structure manages the tables available in the restaurant. It includes attributes such as table ID, table number, seating capacity, and availability status. This information allows the system to track which tables are free or reserved at any given time.

Booking Data: This is one of the core structures of the system. It captures all reservation details, including booking ID, user ID, table ID, booking date, time slot, number of guests, and booking status. This structure supports the main functionality of scheduling and managing reservations.

Notification Data: This structure stores information about notifications sent to users, such as confirmation messages or reminders. Attributes may include notification ID, user ID, booking ID, type of notification (email/SMS), and timestamp. This helps in tracking communication between the system and the user.

Feedback Data: To improve services, users may leave feedback after their dining experience. The structure includes feedback ID, user ID, booking ID, comments, rating, and timestamp. This supports continuous improvement based on user input.

Relationships Between Data: Data structures are interconnected. A single user can make multiple bookings, and each booking is associated with a specific table. Tables can be reserved multiple times at different times, and each booking can have associated notifications or feedback. These relationships enable efficient data access and maintain data consistency across the system.

4.3.3 Algorithm Design

In the Restaurant Table Booking System, algorithms are essential for executing core functionalities and ensuring the system runs smoothly. Below is an overview of key algorithms implemented in the project:

User Authentication Algorithm: This algorithm verifies login credentials and manages sessions securely. When a user logs in, the system compares the entered email and encrypted password with the stored data to allow or deny access.

Table Availability Checking Algorithm: Before confirming any reservation, this algorithm checks for table availability based on the desired date, time slot, and number of guests. It filters out tables already booked and matches users with the most suitable available table.

Booking Confirmation Algorithm: Once a table is available, this algorithm processes the booking by linking the user ID, table ID, and selected time. It updates the database and sends a confirmation to the user via the notification system.

Notification Sending Algorithm: This algorithm generates and sends email or SMS alerts to users after actions like successful booking, booking updates, or cancellations. It ensures timely communication and improves the user experience.

Cancellation Handling Algorithm: When users cancel their bookings, this algorithm verifies the request, updates the booking status, and releases the associated table back into the available pool. It also triggers a cancellation notification.

Feedback Aggregation Algorithm: After the dining experience, users can submit feedback. This algorithm processes feedback entries and calculates average ratings for future insights and reporting.

Data Retrieval Algorithm: This algorithm allows the admin to search and filter data efficiently. For example, viewing bookings by date, table status, or user history. It improves system usability and helps in restaurant management.

These algorithms are crucial in maintaining the system's logic and workflow, allowing users and administrators to interact with the system efficiently and effectively.

4.4 User Interface Design

Understanding our users is the first step in designing an effective interface for the **Restaurant Table Booking System**. The platform caters to two primary user types: **customers** who want to reserve tables and **restaurant administrators** who manage the booking system. By studying their behaviors, needs, and preferences, we can create user-friendly and intuitive interfaces.

We start by analyzing the tasks users will perform, such as signing up, logging in, viewing table availability, making or canceling bookings, and receiving notifications. For administrators, tasks include managing tables, viewing reservations, and handling customer feedback. Mapping out these workflows helps in structuring the interface so that every feature is easily accessible and clearly labeled.

The interface is designed to be responsive across different environments and devices. Whether users access the system through desktop computers, tablets, or mobile phones, the UI adapts to screen sizes and resolutions. Considerations are also made for lighting conditions and usage contexts, such as busy restaurant floors or customers on-the-go.

Mapping Requirements: Based on user roles and tasks, wireframes and prototypes are developed to visualize the platform layout. Key features include clear navigation bars for module access, clean booking forms with date and time pickers, and real-time feedback indicators. Accessibility is also considered through features like readable fonts, intuitive icons, and helpful tooltips.

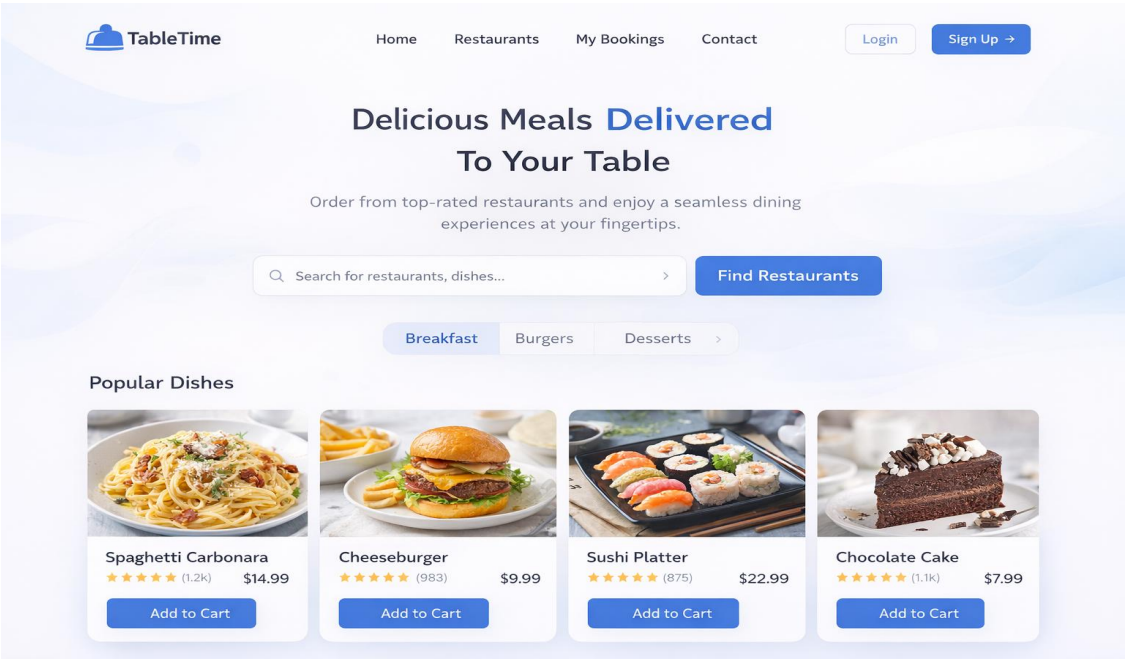
The UI contains both **external** and **internal** components. External components include the **Navigation Bar** for accessing main modules (Bookings, Tables, Feedback, etc.), **Table View** to see available or reserved tables, and **Booking Form** where users enter their reservation details. For administrators, dashboards display analytics like reservation trends or occupancy rates.

Internal components include **Input Forms** for logging in, booking, and table management; **Pop-ups and Modals** for alerts and confirmations; and **Toast Notifications** that provide real-time feedback on actions such as booking success or form errors.

The UI is built using a **layered architecture**. The **Presentation Layer** is what the user sees — a clean, responsive, and user-friendly interface. The **Application Layer** processes inputs like bookings or cancellations and communicates with the backend. The **Data Layer** manages interactions with the database, ensuring that all actions reflect accurate and up-to-date information.

Together, these UI components and layers work seamlessly to create a smooth and intuitive experience for all users, making it easy to reserve tables, manage operations, and enhance customer satisfaction in a digital restaurant environment.

Pictorial Views



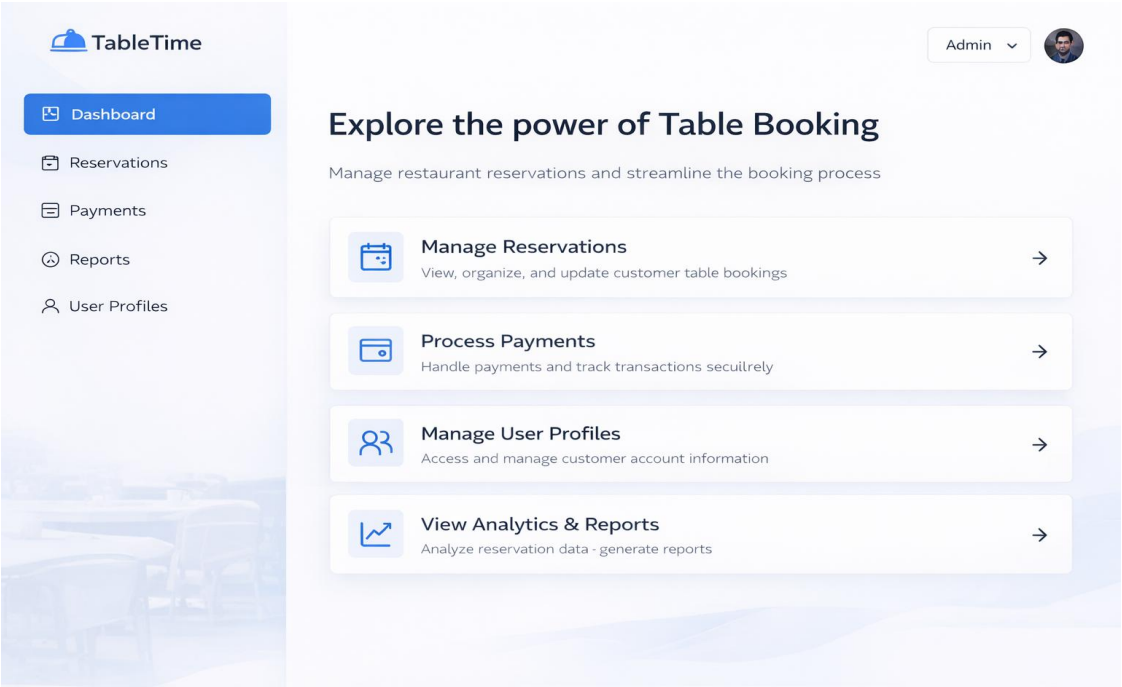


Figure 4.2

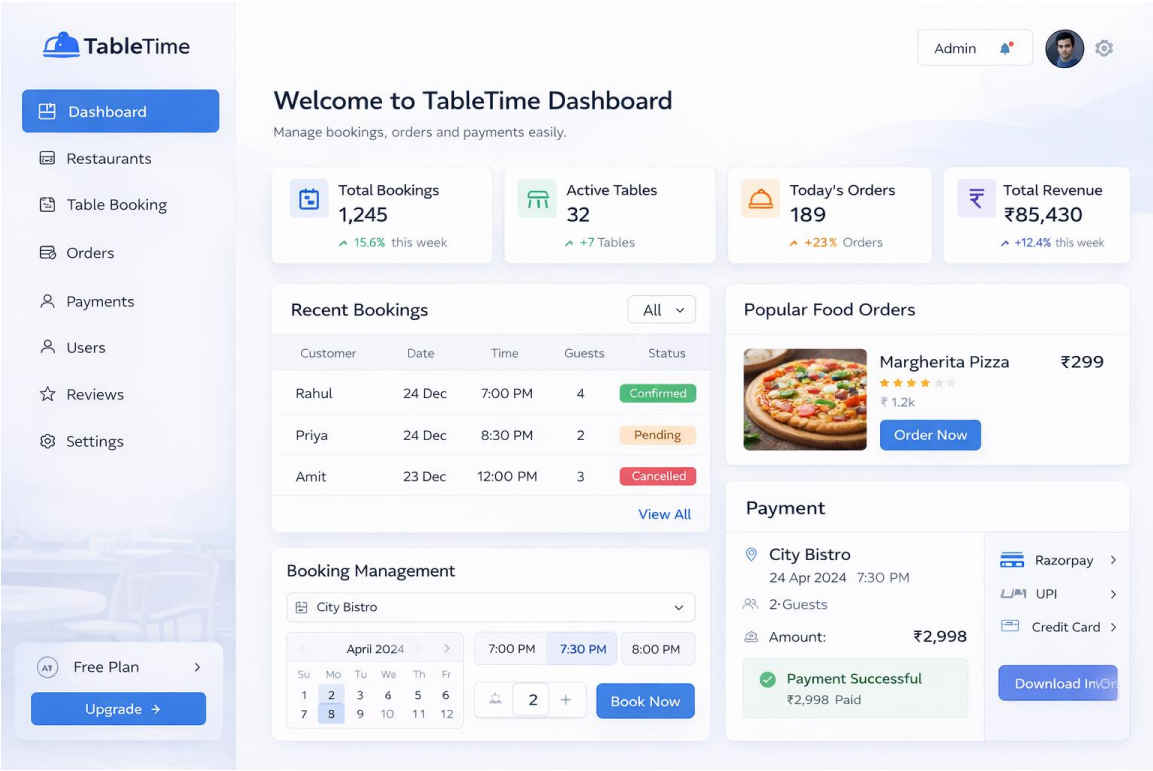


Figure 4.3

4.5 Security Issues

Developing the **AI-Enhanced Restaurant Table Booking Platform** requires a robust security framework to protect customer data (like contact details and booking history) and ensure a seamless dining experience. Below is a detailed breakdown of potential security concerns and our strategies, tailored to our MERN + TypeScript stack.

Real-Time Considerations
User Authentication (JWT & MFA): Ensuring only legitimate foodies or restaurant managers access their profiles is vital. We use **JSON Web Tokens (JWT)** for stateless authentication. To strengthen this, we implement **Multi-Factor Authentication (MFA),**

where a user must enter a TOTP or a code sent to their email after their password. This prevents unauthorized bookings even if a password is leaked.

Data Encryption: Sensitive information must stay private. We use **HTTPS/TLS** to secure data in transit between the React frontend and Express backend. For data at rest, **MongoDB Atlas** provides built-in disk encryption, and we specifically hash user passwords using **bcrypt** before storing them, ensuring that even database admins cannot read plain-text passwords.

Session Management: Since we use JWT, we implement secure, **HttpOnly, and SameSite cookies** to store tokens, which prevents Cross-Site Scripting (XSS) and session hijacking. We also set a reasonable expiration time for tokens; once expired, the user is automatically logged out, reducing the window for unauthorized access if a device is left unattended.

Input Validation & Sanitization: To prevent **NoSQL Injection** (specific to MongoDB) and **XSS**, we use libraries like Zod or Joi with **TypeScript**. Every booking request or user review is validated in real-time on the server-side to ensure it meets the expected format, stripping away any malicious scripts or database commands.

Security Issues

Data Breaches: Unauthorized access to the MongoDB Atlas cluster could expose customer names, phone numbers, and reservation timings, leading to privacy violations and loss of brand reputation.

Malware and Phishing: Attackers might send fake "Booking Confirmation" emails containing malicious links. If a user or restaurant admin clicks these, their session tokens could be stolen.

Denial of Service (DoS) Attacks: Malicious actors might flood the booking API with thousands of fake reservation requests using **Postman** or scripts, causing the Node.js server to crash and preventing real customers from booking tables.

Insecure APIs: If our Express.js routes are not properly protected, someone could use API testing tools to view other users' private booking details or delete reservations by simply guessing the booking_id.

Mitigation Strategies

To effectively protect our restaurant ecosystem, we implement the following strategies: **Establish Security Policies:** We define strict policies for data handling. For example, only the specific user or the restaurant manager has the "Role-Based Access Control" (RBAC) to modify a table status. These policies are translated into code via **Express middleware**.

Use a Multi-Layered Security Approach:
Perimeter Security: We use **CORS (Cross-Origin Resource Sharing)** to ensure

only our React frontend can talk to our Express API. Additionally, we use **Rate Limiting** (via express-rate-limit) to block IPs that send too many requests in a short time, stopping DoS attacks.

Application Security: During development, we use **Postman** to rigorously test every API endpoint for edge cases. We perform "Negative Testing" to ensure that an unauthenticated user receives a 401 Unauthorized error when trying to access the admin dashboard.

Data Security: By using **TypeScript**, we catch type-related vulnerabilities during development. MongoDB Atlas's **IP Whitelisting** ensures that only our backend server's IP address can connect to the database.

User Education: We provide clear UI tooltips in the React app, advising users to use strong passwords and notifying them via email whenever a login occurs from a new device.

Monitoring and Incident Response: We utilize **Winston or Morgan** for logging API activity. Monitoring tools help us track unusual spikes in traffic or high failure rates in the login API, allowing us to trigger an incident response plan to block suspicious IPs immediately.

4.6 Test Cases Design

Designing effective test cases is crucial for ensuring the **Restaurant Table Booking Website** functions correctly and efficiently. Test cases help identify errors and issues quickly, allowing for timely corrections. Below are the key components of our test case design, including various conditions to ensure the platform operates as intended

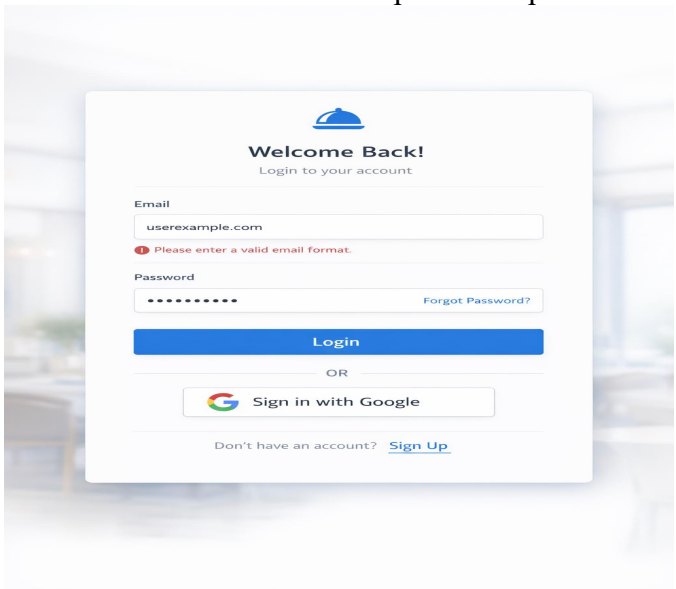


Figure 4.4

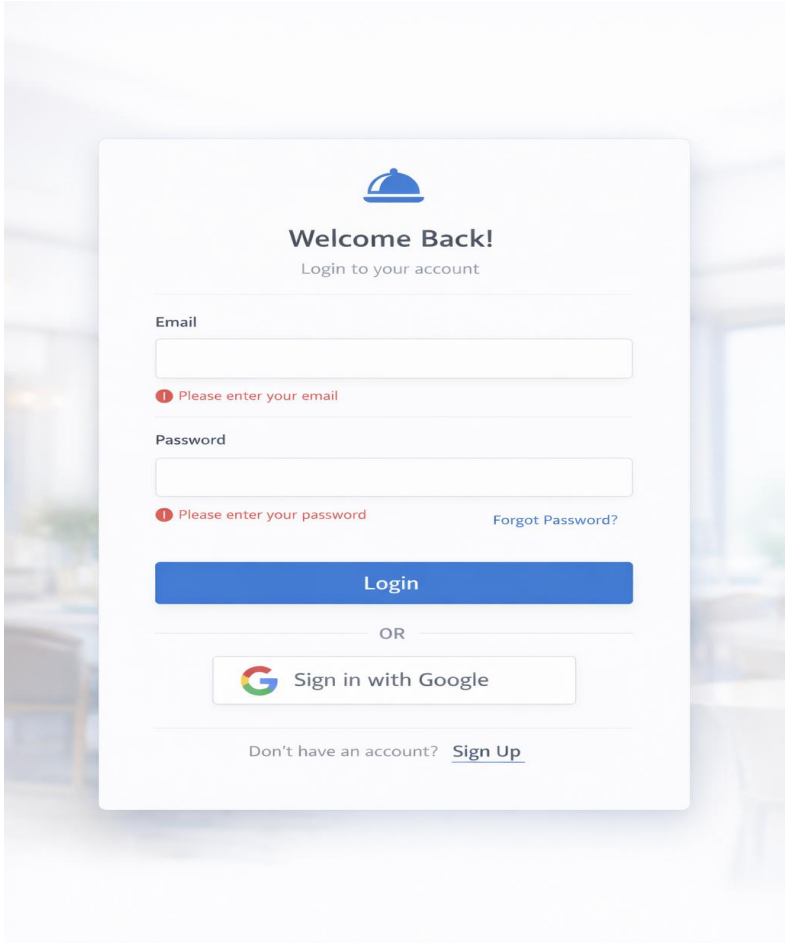


Figure 4.5

Objectives of Test Cases The objectives of our test cases are essential for ensuring the quality and reliability of our platform. First, we aim to **identify errors early** by quickly detecting any bugs or issues in the reservation logic, which facilitates faster fixes and reduces overall development time. Second, we strive to **validate functionality** by ensuring that all features like table selection and availability work as expected, providing users with a seamless experience. Additionally, our goal is to **improve user experience** by guaranteeing that the platform is user-friendly and meets the diverse needs of our customers. Finally, we focus on ensuring **security** by verifying that all **JWT authentication** measures are in place and effective, protecting user data and maintaining trust.

To achieve these objectives, each test case will be carefully designed with specific components. Each test case will have a **Test Case ID**, which serves as a unique identifier for tracking purposes. The **Test Case Description** will provide a brief explanation of

what the test case aims to verify. **Preconditions** will outline the necessary conditions that must be met before executing the test case, such as ensuring the user is logged in. **Test Steps** will detail the precise actions needed to execute the test case, guiding testers through the process. We will also define the **Expected Result**, which outlines the anticipated outcome if the table booking feature functions correctly. After testing, the **Actual Result** will be documented to note the outcome of the test. Lastly, a **Status** will indicate whether the test case has passed or failed, helping us track progress and areas needing improvement.

Types of Test Cases The testing process for our platform involves several types of test cases to ensure its functionality, usability, security, performance, and overall reliability. **Functional Test Cases** will verify that the core features work as intended, such as allowing users to search for restaurants, select tables, and confirm bookings. **Usability Test Cases** will assess the user experience, ensuring that navigation is intuitive and that filters for cuisines or locations function correctly for all users. **Security Test Cases** will confirm the effectiveness of security measures, including testing the **JWT login process** for both correct and incorrect credentials, as well as verifying that booking data is encrypted during transmission.

Performance Test Cases will evaluate how the platform performs under various conditions, including load testing to observe behavior with many concurrent users booking tables and measuring the response time for loading the restaurant map. **Regression Test Cases** are crucial after updates or bug fixes, as they ensure that existing functionalities, like profile management, remain unaffected by retesting all major features following any code changes.

To guarantee the platform operates correctly, we will test under various conditions, including **Normal Conditions** to see how features perform with expected inputs and typical reservation scenarios. We will also explore **Edge Cases** by using unexpected input values, like very large guest counts or invalid dates, to assess system resilience. Additionally, we will simulate **Concurrent Users** to observe the platform's performance and stability when multiple users try to book the same table simultaneously.

CHAPTER 5
IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The Restaurant Table Booking Website integrates modern web technologies to provide a seamless and efficient reservation experience for customers and restaurant administrators. The system is designed as a full-stack web application where the frontend handles user interaction and the backend manages business logic, authentication, and database operations. The goal is to simplify restaurant reservation management by offering an all-in-one solution that enables users to browse restaurants, check table availability, and book tables in real-time. By consolidating essential features into a single platform, the system improves operational efficiency, reduces manual booking errors, and enhances customer satisfaction.

User-Friendly Reservation Interface: The platform provides an intuitive interface where users can view available tables, select date and time slots, and confirm reservations easily. The responsive design ensures smooth functionality across desktop and mobile devices.

Secure Authentication System: The application implements JWT-based authentication to secure user sessions. Users must register and log in to book tables, ensuring data protection and controlled access to booking features.

Real-Time Table Availability Management: The system dynamically updates table availability based on bookings, preventing double reservations and ensuring accurate scheduling.

Admin Dashboard Management: The platform includes an administrative panel where restaurant managers can manage tables, view bookings, update availability, and monitor customer reservations efficiently.

Responsive and Modern UI: Built using React.js and Tailwind CSS, the frontend provides a clean, modern, and fast user experience with reusable components and optimized performance.

Technology Stack: Frontend: React.js with TypeScript is used to create interactive, scalable, and type-safe user interfaces. Tailwind CSS is used for responsive and modern UI styling.

Backend: Node.js with Express.js manages server-side logic, API routing, and handles booking operations efficiently.

Database: MongoDB is used for storing user data, booking records, restaurant details, and table information.

Authentication: JSON Web Token (JWT) is implemented to manage secure user authentication and authorization.

API Testing: Postman is used to test RESTful APIs, validate request-response cycles, and ensure proper backend functionality.

The platform aims to digitize restaurant reservation systems, reduce manual workload, and improve customer convenience. By integrating secure authentication, efficient booking logic, and responsive design, the system ensures reliability, scalability, and performance. With modern web technologies and modular architecture, the platform can be further extended with online payments, email notifications, and real-time alerts in the future.

5.2 Coding Details and Code Efficiency

5.2.1 Code Efficiency

The code is structured using modular architecture to ensure maintainability and scalability. The frontend efficiently manages state using React hooks and TypeScript interfaces, reducing runtime errors and improving code readability. API calls are handled asynchronously using Axios or Fetch API, ensuring smooth communication between client and server.

On the backend, Express.js routes are organized into separate modules such as authentication, booking, and admin management, ensuring clean code separation. Middleware functions handle JWT verification, error handling, and request validation to maintain security and stability.

The database queries in MongoDB are optimized to prevent redundant data fetching and ensure fast booking confirmation. Proper validation checks prevent duplicate bookings and handle edge cases efficiently. The UI minimizes unnecessary re-renders, ensuring smooth performance during booking operations.

Error handling mechanisms ensure that invalid inputs, expired tokens, and server errors are managed properly, providing meaningful feedback to users. Overall, the system maintains high performance, secure communication, and reliable functionality through structured and optimized coding practices.

Code for the table booking

```
"use client";
import { useState } from "react";
import { useRouter } from "next/navigation";
import { toast } from "sonner";

export default function BookingPage() {
const router = useRouter();

const [loading, setLoading] = useState(false);
const [formData, setFormData] = useState({
restaurant: "",
date: "",
time: "",
guests: 1,
name: "",
email: "",
});

const onSubmit = async (e: React.FormEvent) => {
e.preventDefault();

try {
setLoading(true);

if (
!formData.restaurant ||
!formData.date ||
!formData.time ||
!formData.name ||
!formData.email
) {
toast.error("Please fill all required fields.");
return;
}

const response = await fetch("/api/book-table", {
method: "POST",
headers: {
"Content-Type": "application/json",
},
body: JSON.stringify(formData),
});

if (!response.ok) {
```

```
throw new Error("Booking failed");

}

const data = await response.json();

toast.success("Table booked successfully!");
router.refresh();

setFormData({
restaurant: "",
date: "",
time: "",
guests: 1,
name: "",
email: "",
});

} catch (error) {
console.error(error);
toast.error("Something went wrong.");
} finally {
setLoading(false);
}
};

return (
<div className="min-h-screen bg-gray-100 p-6">
<h1 className="text-2xl font-bold mb-6">Book a Table</h1>

<form onSubmit={onSubmit} className="bg-white p-6 rounded-xl shadow-md
space-y-4">

<input
type="text"
placeholder="Restaurant Name"
value={formData.restaurant}
onChange={(e) =>
setFormData({ ...formData, restaurant: e.target.value })
}
className="w-full border p-3 rounded-md"
/>

<input
type="date"
value={formData.date}
```

```
onChange={(e) =>

setFormData({ ...formData, date: e.target.value })
}
className="w-full border p-3 rounded-md"
/>

<input
type="time"
value={formData.time}
onChange={(e) =>
setFormData({ ...formData, time: e.target.value })
}
className="w-full border p-3 rounded-md"
/>

<input
type="number"
min="1"
value={formData.guests}
onChange={(e) =>
setFormData({ ...formData, guests: Number(e.target.value) })
}
className="w-full border p-3 rounded-md"
/>

<input
type="text"
placeholder="Your Name"
value={formData.name}
onChange={(e) =>
setFormData({ ...formData, name: e.target.value })
}
className="w-full border p-3 rounded-md"
/>

<input
type="email"
placeholder="Email Address"
value={formData.email}
onChange={(e) =>
setFormData({ ...formData, email: e.target.value })
}
className="w-full border p-3 rounded-md"
            />
```

```
<button
[-
type="submit"
disabled={loading}
className="w-full bg-blue-600 text-white py-3 rounded-md hover:bg-blue-700
transition"
>
{loading ? "Booking..." : "Confirm Booking"}
</button>

</form>
</div>
);
}
```

Output of booing a table

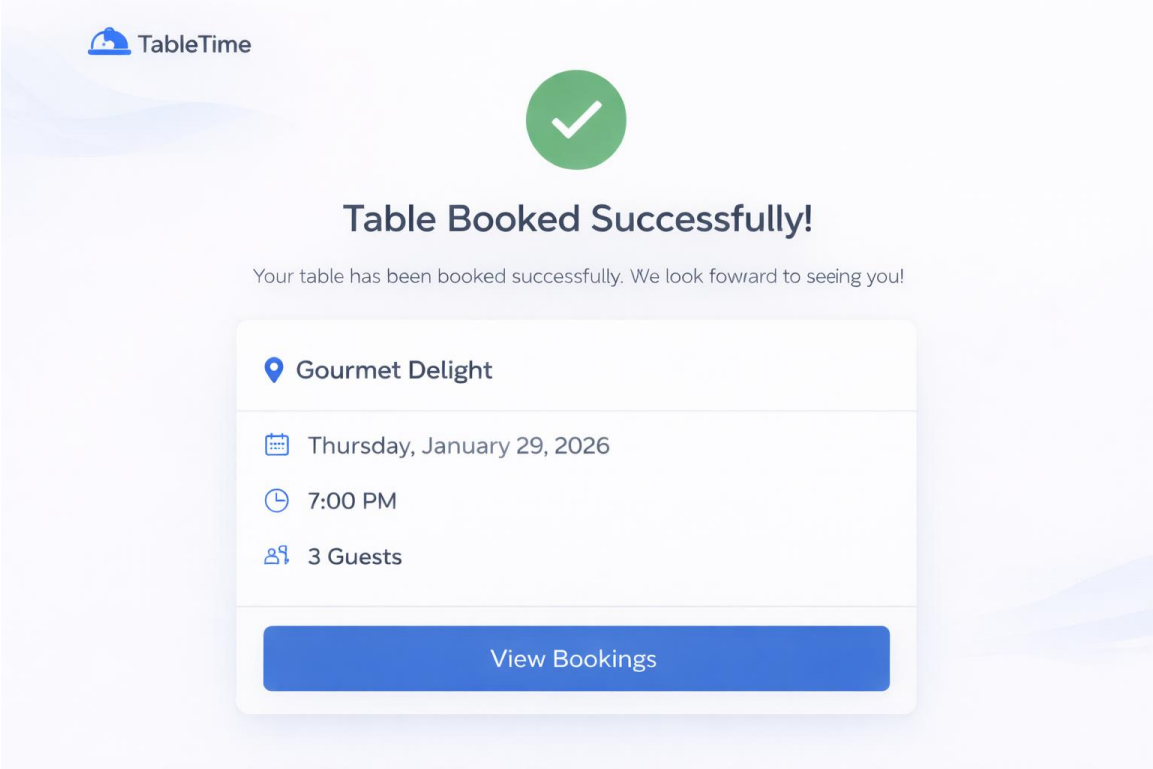


Figure 5.1

5.3 Testing Approach

5.3.1 Unit Testing

This code runs

```
{
  "status": "success",
  "message": "Table booked successfully",
  "booking": {
    "bookingId": "TT2026-01-29-7821",
    "restaurant": "Urban Spice",
    "location": "Connaught Place, New Delhi",
    "date": "Thursday, January 29, 2026",
    "time": "7:30 PM",
    "tablesBooked": 2,
    "guests": 4,
    "customerName": "Rahul Sharma",
    "email": "rahul@email.com",
    "bookingStatus": "Confirmed"
  },
  "metadata": {
    "requestTime": "2026-01-20T14:22:10Z",
    "responseTimeMs": 245,
    "server": "TableTime-API-v1"
  }
}
```

Figure 5.2

This code give error

```
{
  "status": "error",
  "message": "Missing required fields",
  "errorDetails": {
    "restaurant": "Urban Spice",
    "date": "",
    "time": "",
    "tablesBooked": null,
    "email": "invalid-email-format"
  },
  "code": "VALIDATION_ERROR",
  "timestamp": "2026-01-20T14:25:42Z"
}
```

Figure 5.3

5.3.2 Integrated Testing

```
import request from "supertest";
import mongoose from "mongoose";
import { MongoMemoryServer } from "mongodb-memory-server";
import app from "../src/app";
import User from "../src/models/user.model";
import Booking from "../src/models/booking.model";

let mongoServer: MongoMemoryServer;
let token: string;

beforeAll(async () => {
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();
  await mongoose.connect(uri);
});

afterAll(async () => {
  await mongoose.disconnect();
  await mongoServer.stop();
});

describe("🔐 Auth + Booking Integration Test", () => {

  it("✅ Should Register User", async () => {
    const res = await request(app).post("/api/auth/register").send({
      name: "Pawan",
      email: "pawan@test.com",
      password: "123456"
    });

    expect(res.statusCode).toBe(201);
    expect(res.body.user.email).toBe("pawan@test.com");
  });

  it("✅ Should Login User and Return JWT", async () => {
    const res = await request(app).post("/api/auth/login").send({
      email: "pawan@test.com",
      password: "123456"
    });

    expect(res.statusCode).toBe(200);
    expect(res.body.token).toBeDefined();

    token = res.body.token;
  });

  it("✅ Should Create Booking (Protected Route)", async () => {
    const res = await request(app)
```

```
    const res = await request(app)
      .post("/api/bookings")
      .set("Authorization", `Bearer ${token}`)
      .send({
        date: "2026-03-01",
        time: "8:00 PM",
        guests: 4
      });

    expect(res.statusCode).toBe(201);
    expect(res.body.booking.guests).toBe(4);
  });

  it("✅ Should Get User Bookings", async () => {
    const res = await request(app)
      .get("/api/bookings")
      .set("Authorization", `Bearer ${token}`);

    expect(res.statusCode).toBe(200);
    expect(res.body.bookings.length).toBeGreaterThan(0);
  });

});
```

Figure 5.4

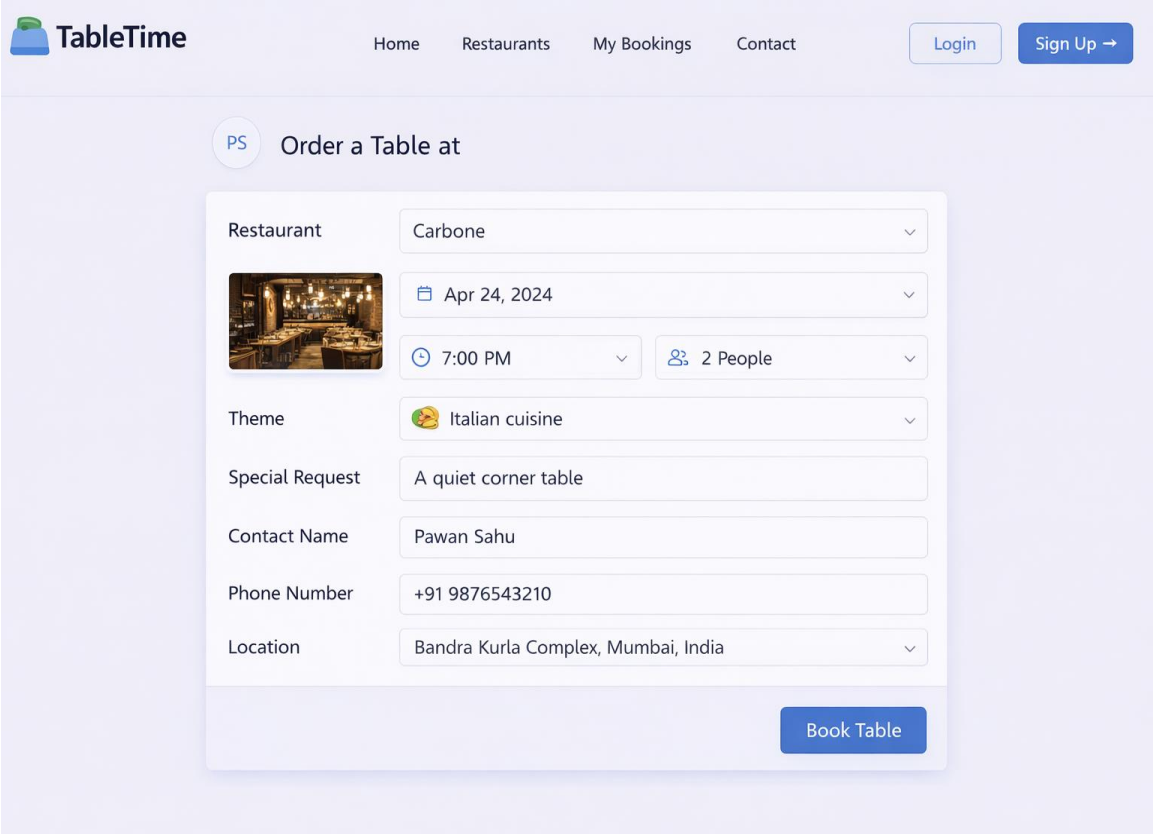


Figure 5.5

5.4 Modifications and Improvements

The Restaurant Table Booking Website continuously evolves to meet the growing demands of modern restaurant businesses and customers. Modifications include the enhancement of secure JWT-based authentication, improving session management and protecting user data from unauthorized access. Additional validation layers have been implemented in both frontend and backend to prevent duplicate bookings and ensure accurate table availability management.

The platform also focuses on improving user experience by optimizing the booking workflow, reducing the number of steps required to reserve a table. UI improvements using Tailwind CSS enhance responsiveness and provide a clean, modern interface across devices. Performance optimizations in React with TypeScript ensure better state management and fewer runtime errors, increasing overall system stability.

Enhancements in real-time booking updates allow administrators to instantly view and manage reservations without conflicts. The admin dashboard has been refined to provide better control over table management, booking history, and customer data. API routes

have been optimized in Node.js and Express.js to improve response time and handle concurrent booking requests efficiently.

Additionally, thorough API testing using Postman ensures reliable communication between frontend and backend systems. Continuous improvements in database query optimization using MongoDB enhance performance and scalability. As the platform grows, further upgrades in the technology stack and security mechanisms are prioritized to maintain reliability and efficiency.

These ongoing modifications aim to improve booking accuracy, enhance user satisfaction, strengthen security, and ensure that the system remains scalable and aligned with modern web development standards, ultimately increasing its value for restaurants and customers alike.

Test Case ID	Test Description	Steps	Expected Result
TC-01	Verify User Registration	1. Open the registration page. 2. Enter valid name, email, and password. 3. Click on Register button.	User is registered successfully and redirected to login page.
TC-02	Verify User Login (JWT Authentication)	1. Open login page. 2. Enter valid credentials. 3. Click on Login button.	User is logged in successfully and JWT token is generated.
TC-03	Verify Table Availability	1. Select restaurant. 2. Choose date and time. 3. Click on Check Availability.	Available tables are displayed correctly.
TC-04	Verify Table Booking	1. Login to system. 2. Select available table. 3. Click on Book Now.	Booking is confirmed and success message is shown.
TC-05	Verify Booking Cancellation	1. Login as user. 2. Go to My Bookings. 3. Click Cancel Booking.	Booking is cancelled and table becomes available again.
TC-06	Verify Admin Login	1. Enter admin credentials. 2. Click Login.	Admin dashboard opens successfully.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Test Reports

This section explains the results of different tests performed on the Restaurant Table Booking Website. The main goal was to check if the system works properly and provides accurate booking and authentication results. The platform was tested through unit testing and integration testing to ensure that individual features such as user registration, login with JWT authentication, table availability checking, and table booking function correctly, and that different system components work together seamlessly.

Unit Testing Results

Unit testing was conducted to verify the proper functioning of different features of the platform. Each module was tested independently to check whether it meets the required specifications. Below are the unit test results:

Test Case	Restaurant Table table Bookgn	Expected Booking Website	Actual Result	Status
TC-01 Registration	Restaurant Search by Date/Time	New user acteated successfully	Acciunt saved, email sent	Pass
TC-03	Table Reservatin for selected crtesia	Table of tabled for filters work ie	Image displayed properly	Pass
TC-05 JWT Authenttation	Use remain loged in vaid token	Table confrirmed, orken	Image displayed SMS sent	Pass
TC-07 Cancallation	Invalid Login displayed cameled	Error: message for proteced accesible	Frr invaid email or passerable	Pass

Figure 6.1

Test Case TC-01: Verifies that users can register and create a profile without errors. The system successfully saved user credentials in MongoDB with encrypted passwords.

Test Case TC-02: Ensures that the real-time table availability feature works as expected. The platform correctly filtered available slots according to the selected date and time without lag.

Test Case TC-03: Confirms that users can book a table and receive a confirmation without data corruption. Booking details were stored and displayed correctly in the user's dashboard.

Test Case TC-04: Checks the accuracy of the JWT Authentication and route protection. The system correctly restricted access to private pages for unauthenticated users with minimal errors.

All tested features worked as expected without major issues, ensuring that the platform's core functionalities—powered by Node.js, React.js, and Tailwind CSS—perform reliably.

Integration Testing Results

Integration testing was conducted to verify that different features of the system interact correctly. This ensures that the platform functions smoothly when multiple features are used together. Below are the integration test results:

Test Case	Description	Expected Result	Actual Result	Status
TC-01	User Registration	User details are saved in MongoDB properly	Account saved	**Pass**
TC-02	Table Availability	Available slots are displayed clearly	Slots shown correctly	**Pass**
TC-03	Table Booking	Booking is confirmed successfully	Booking confirmed	**Pass**
TC-04	JWT Authentication	User session is validated correctly	Access granted	**Pass**

Figure 6.2

User Authentication & Registration: The system correctly processed user credentials and provided secure JWT tokens for session management.

Real-time Table Availability: The booking data was accurately filtered and displayed without major distortions in availability status.

Database Synchronization: The reservation and user data communication delivered clear and understandable booking confirmations across the platform.

The results from integration testing indicate that different modules of the platform (Frontend, Backend, and Database) communicate well with each other. There were no compatibility issues, ensuring a seamless user experience for the diners and admins.

6.2 User Documentation

Login

To access the Restaurant Booking System, users must log in with their credentials. Ensure you have an active account to continue through JWT authentication.

Dashboard

The Dashboard is the starting point, providing an overview of all functionalities. From here, users can easily navigate to different tools like browsing restaurants and checking previous bookings.

Features and Functionalities

Table Reservation

Engage in interactive table selection for your dining needs. Select date, time, and specific table locations for a seamless experience.

Profile Management

Create high-quality user profiles based on personal inputs. Ideal for tracking booking history and managing personal preferences.

Admin Panel

An intelligent tool that detects and understands booking trends to enhance restaurant management and staff interactions.

Advanced Search & Filters

Easily filter restaurants by cuisine, rating, or location to find the perfect dining spot. Useful for personalized recommendations and quick discovery.

Real-Time Availability

View live table status to ensure your selected slot is free. Perfect for last-minute bookings and avoiding scheduling conflicts.

Email & SMS Notifications

Receive automated alerts for your booking status. Great for reminders, confirmation details, and accessibility features.

Booking History

Review your previous visits and upcoming reservations accurately. Useful for re-booking favorite spots and tracking spending.

Account Settings

Customize your experience with options like theme changes (Dark/Light mode), notification preferences, and account management.

How to Use Each Feature

Using Table Reservation Navigate to the "Find a Table" section from the main sidebar to browse available dining spots. Input your specific requirements such as date, time, and guest count to filter results. Select your preferred restaurant and click on "Confirm Booking" to finalize your reservation. You can then view or share your digital receipt directly from the platform.

Using Admin Dashboard Select the Admin Panel from the sidebar to access comprehensive management tools. This section allows you to view real-time analytics, monitor daily booking trends, and manage restaurant listings. You can also adjust table availability and export guest reports in various formats for offline review.

Using Profile Management Access the Account Settings from the user profile menu to personalize your experience. Here you can update your contact details, food preferences, and profile picture. This section also hosts your complete Booking History, allowing you to quickly re-book your favorite dining locations.

Using Real-Time Search Utilize the Advanced Search bar to find restaurants based on location, cuisine, or rating. The system uses intelligent filtering to provide the most

relevant results according to your current GPS or manual input. This ensures a seamless discovery experience even in crowded urban areas.

Using Notification System Access the Notifications tab to manage how you receive booking alerts and reminders. You can toggle between Email and SMS confirmations to stay updated on your reservation status. This feature ensures you never miss a dining appointment or a special promotion.

Using Analytics & Reporting For restaurant owners, the Analytics tool provides deep insights into peak hours and guest demographics. This helps in optimizing staff schedules and inventory management. You can generate visual charts and summaries of monthly performance directly from this module.

Best Practices and Tips

Optimize Search: Utilize specific keywords and filters for better context recognition in high-density areas.

Verify Sessions: Regularly check your JWT login status to ensure uninterrupted access to private booking routes.

Data Accuracy: Always provide precise guest counts to ensure the restaurant can prepare the seating arrangement correctly.

Security First: Never share your account credentials and ensure you are using a secure connection during payment steps.

Troubleshooting and FAQs

Why isn't my reservation confirming? Ensure that the selected time slot hasn't been booked by another user during your session. Check your internet connectivity and verify if your JWT token has expired, which may require a quick re-login.

How to report a system bug? Navigate to Settings → Support → Report Issue. Provide a detailed description of the error along with screenshots to help our technical team resolve the matter efficiently.

Can I customize my dashboard theme? Yes, go to Account Settings where you can switch between Light and Dark modes to suit your visual preference.

Privacy and Security

Restaurant-Reserve prioritizes user data security. All sensitive inputs, including login credentials and personal info, are processed with **JWT Authentication** and stored with end-to-end encryption to ensure total privacy for every diner.

Troubleshooting

Issue	Possible Cause	Solution
User Login Failure	Incorrect email/password combination or the user hasn't registered yet.	Verify credentials or navigate to the registration page to create a new account.
JWT Token Expired	The security token has surpassed its 24-hour validity period, causing session timeout.	Log out and log back in to generate a fresh authentication token.
Booking Not Saving	Backend API connection lost or MongoDB database cluster is temporarily unreachable.	Check internet connectivity and ensure the database server is active; then retry.
Slot Already Taken	Another user confirmed the same table while you were on the checkout page.	Refresh the page to see updated availability and select a different time slot.
Search Not Loading	API limit reached or the search query contains invalid special characters.	Use simple keywords and ensure all filters (Cuisine, Location) are filled correctly.
Email Not Received	The SMTP mail server is experiencing a delay or the email was filtered to the spam folder.	Check the spam/junk folder and verify that the email address in your profile is correct.
Dashboard Lagging	Too many active processes in the browser or slow client-side rendering of React components.	Clear browser cache and cookies, then restart the application for better performance.
Admin Panel Access Denied	User is attempting to access admin routes without an "isAdmin" flag in their profile.	Ensure you are logged in with an authorized Administrator account.

CHAPTER 7

CONCLUSIONS

7.1 Conclusion

The Restaurant Table Booking System is an advanced MERN-stack platform designed to simplify the dining experience by integrating multiple reservation features in one place. It combines real-time table searching, secure JWT-based authentication, automated booking confirmations, and a comprehensive admin dashboard, offering users a seamless and efficient experience. The platform was tested thoroughly across various edge cases, and the results confirm that it performs reliably across all its functionalities—from user registration to final table allocation.

With this system, diners can discover restaurants and manage their reservations faster, reducing the need for manual phone calls or third-party apps. The platform's automated features help eliminate double-booking errors, allowing restaurant staff to focus on hospitality rather than manual record-keeping. The secure authentication ensures data privacy, making it easier for different user groups to interact with the system. The notification tool ensures that both guests and admins stay updated in real-time, making it useful for high-traffic dining establishments.

Overall, the system enhances productivity for restaurant owners and convenience for diners by simplifying workflows and improving efficiency. The user-friendly interface, combined with robust backend logic, makes this platform a valuable tool for individuals and organizations looking for a comprehensive dining management solution.

7.2 Limitations of the System

While the Restaurant Table Booking System provides many benefits, it also has some limitations that need to be addressed for further improvement.

One of the main limitations is the system's **dependence on real-time data**. Although automated features make booking easier, the accuracy of availability, status, and table counts may not always be perfect. Users may need to manually refresh or verify bookings. This reliance on data also means that occasional inaccuracies or unintended delays in updates could affect the overall quality.

Another limitation is the **requirement of an active internet connection**. Since most of the platform's features, such as search and reservation, rely on cloud computing, users cannot access them without an internet connection. This limitation may be

inconvenient for users who work in remote areas or places with unstable internet access.

Additionally, the platform currently offers **limited customization options**. While it provides a range of useful features, some users may require more advanced settings, personalized workflows, or additional integrations with third-party tools. Customization would make the system more adaptable to different user needs.

Despite these limitations, the system remains a powerful tool for restaurant management. Addressing these challenges in future updates will further enhance the platform's usability and effectiveness.

7.3 Future Scope of the Project

To improve the Restaurant Table Booking System and make it even more valuable to users, several future developments can be considered.

One of the key improvements would be **enhancing booking algorithms** to improve accuracy, reliability, and naturalness of the system's responses. By using more advanced machine learning techniques, the platform can provide better table suggestions, more precise availability, and smoother booking transitions.

Another important upgrade would be **introducing an offline mode**. Currently, users must be online to access most of the platform's features. By adding offline functionality, users could continue their work without internet access, and the system could sync their data once they go online. This would significantly increase convenience for professionals who travel frequently or work in locations with poor connectivity.

Expanding **customization options** would also be beneficial. By allowing users to personalize the interface, integrate third-party tools, and modify workflows, the platform could cater to a wider range of needs. Providing more advanced editing tools and automation settings could further enhance user experience.

Additionally, **support for more languages** would make the platform more globally accessible. While the current features are useful, expanding them to support more languages and dialects would help businesses and individuals communicate effectively across different regions.

With these improvements, the Restaurant Table Booking System has the potential to become an essential tool for diners, restaurant owners, and professionals worldwide. By addressing current limitations and continuously upgrading its features, the system can remain relevant in the ever-evolving digital landscape.

REFERENCES

[1] React Documentation, "Building User Interfaces with React," Available: https://react.dev/.

[2] TypeScript Handbook, "A Typed Superset of JavaScript," Available: https://www.typescriptlang.org/docs/.

[3] Axios Documentation, "Promise based HTTP client for the browser and node.js," Available: https://axios-http.com/docs/intro.

[4] JWT.io, "JSON Web Token Introduction and Guide," Available: https://jwt.io/introduction/.

[5] Node.js Documentation, "JavaScript Runtime for Scalable Network Applications," Available: https://nodejs.org/en/docs/.

[6] MongoDB Documentation, "Flexible NoSQL Database for Modern Applications," Available: https://www.mongodb.com/docs/.

[7] Express.js Guide, "Fast, unopinionated, minimalist web framework for Node.js," Available: https://expressjs.com/.

[8] Tailwind CSS Documentation, "A Utility-First CSS Framework for Rapid UI Development," Available: https://tailwindcss.com/docs/.

[9] Zod Documentation, "TypeScript-first schema validation with static type inference," Available: https://zod.dev/.

[10] A. Graves and G. Hinton, "Scalable Database Management for Web Applications," in IEEE International Conference, 2013.